Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 1

по дисциплине «Информационные системы»

Информационная система для управления объектами класса Organization

Вариант _____

Выполнил:
студент группы _____
_____

Преподаватель:
_____

Санкт-Петербург
2026

1. Текст задания

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса `Organization`, описание которого приведено ниже:

```
public class Organization {
    private long id; //Значение поля должно быть больше 0, Значение этого поля должно
быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.ZonedDateTime creationDate; //Поле не может быть null, Значение
этого поля должно генерироваться автоматически
    private Address officialAddress; //Поле может быть null
    private Long annualTurnover; //Поле может быть null, Значение поля должно быть
больше 0
    private long employeesCount; //Значение поля должно быть больше 0
    private Float rating; //Поле может быть null, Значение поля должно быть больше 0
    private String fullName; //Поле не может быть null
    private OrganizationType type; //Поле не может быть null
    private Address postalAddress; //Поле может быть null
}
public class Coordinates {
    private double x; //Максимальное значение поля: 549
    private double y;
}
public class Address {
    private String zipCode; //Поле не может быть null
}
public enum OrganizationType {
    GOVERNMENT,
    TRUST,
    PRIVATE_LIMITED_COMPANY;
}
```

Разработанная система должна удовлетворять следующим требованиям:

- Основное назначение информационной системы — управление объектами, созданными на основе заданного в варианте класса.
- Необходимо, чтобы с помощью системы можно было выполнить следующие операции с объектами: создание нового объекта, получение информации об объекте по ИД, обновление объекта (модификация его атрибутов), удаление объекта. Операции должны осуществляться в отдельных окнах (интерфейсах) приложения. При получении информации об объекте класса должна также выводиться информация о связанных с ним объектах.
- При создании объекта класса необходимо дать пользователю возможность связать новый объект с объектами вспомогательных классов, которые могут быть связаны с созданным объектом и уже есть в системе.

- Выполнение операций по управлению объектами должно осуществляться на серверной части (не на клиенте), изменения должны синхронизироваться с базой данных.
- На главном экране системы должен выводиться список текущих объектов в виде таблицы (каждый атрибут объекта — отдельная колонка в таблице). При отображении таблицы должна использоваться пагинация (если все объекты не помещаются на одном экране).
- Нужно обеспечить возможность фильтровать/сортировать строки таблицы, которые показывают объекты (по значениям любой из строковых колонок). Фильтрация элементов должна производиться только по полному совпадению.
- Переход к обновлению (модификации) объекта должен быть возможен из таблицы с общим списком объектов и из области с визуализацией объекта (при ее реализации).
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться в интерфейсах у других пользователей, авторизованных в системе.
- Если при удалении объекта с ним связан другой объект, связанные объекты должны удаляться.
- Пользователи должны иметь возможность просмотра всех объектов. Для модификации объекта должно открываться отдельное диалоговое окно. При вводе некорректных значений в поля объекта должны появляться информативные сообщения о соответствующих ошибках.

В системе должен быть реализован отдельный пользовательский интерфейс для выполнения специальных операций над объектами:

- Вернуть количество объектов, значение поля type которых равно заданному.
- Вернуть количество объектов, значение поля fullName которых больше заданного.
- Вернуть массив объектов, значение поля type которых меньше заданного.
- Добавить нового сотрудника в организацию с указанным id.
- Реализовать поглощение одной организацией другой без увольнения сотрудников.

Представленные операции должны быть реализованы в качестве функций БД, которые необходимо вызывать из уровня бизнес-логики приложения.

Особенности хранения объектов, которые должны быть реализованы в системе:

- Организовать хранение данных об объектах в реляционной СУБД (PostgreSQL). Каждый объект, с которым работает ИС, должен быть сохранен в базе данных.
- Все требования к полям класса (указанные в виде комментариев к описанию классов) должны быть выполнены на уровне ORM и БД.
- Для генерации поля id использовать средства базы данных.
- Для подключения к БД на кафедральном сервере использовать хост pg, имя базы данных — studs, имя пользователя/пароль совпадают с таковыми для подключения к серверу.

При создании системы нужно учитывать следующие особенности организации взаимодействия с пользователем:

- Система должна реагировать на некорректный пользовательский ввод, ограничивая ввод недопустимых значений и информируя пользователей о причине ошибки.
- Переходы между различными логически обособленными частями системы должны осуществляться с помощью меню.
- При добавлении/удалении/изменении объекта, он должен автоматически появиться/исчезнуть/измениться на области у всех других клиентов.

При разработке ИС должны учитываться следующие требования:

- В качестве основы для реализации ИС необходимо использовать Spring MVC.
- Для создания уровня хранения необходимо использовать Hibernate.
- Разные уровни приложения должны быть отделены друг от друга, разные логические части ИС должны находиться в отдельных компонентах.

2. UML-диаграммы

Серверная часть написана на Java 17 с использованием Spring Boot 3.5 (Spring MVC, Spring WebSocket) и Hibernate 6 (через Spring Data JPA), данные хранятся в PostgreSQL. Клиентская часть — набор HTML-страниц с JavaScript-модулями, которые раздаёт само приложение. Диаграммы построены в PlantUML, их исходные описания лежат в каталоге `docs/uml` репозитория. Геттеры, сеттеры и конструкторы, создаваемые внедрением зависимостей, на диаграммах не показаны.

2.1. Диаграмма пакетов

Приложение разделено на уровни: представление (`web`), бизнес-логика (`service`) и хранение (`repository`). Зависимости направлены только «сверху вниз»: контроллеры обращаются к сервисам, сервисы — к репозиториям, а сущности предметной области (`domain`) не зависят ни от одного уровня. Рассылка изменений клиентам вынесена в отдельный пакет `realtime`.

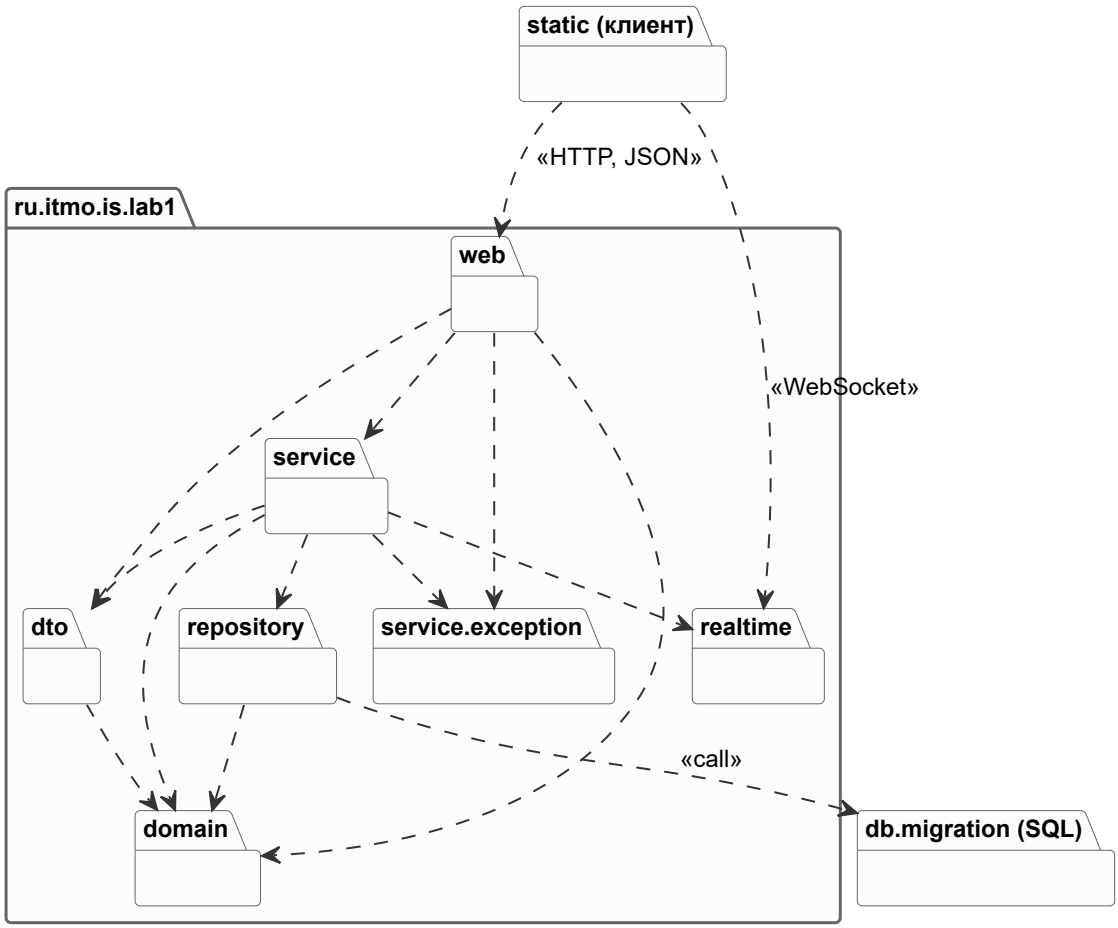


Рисунок 1 — Диаграмма пакетов приложения

Таблица 1 — Назначение пакетов

Пакет	Назначение
`web`	REST-контроллеры Spring MVC и единая обработка ошибок с сообщениями по полям
`dto`	Объекты запросов и ответов REST API (Java records) с аннотациями Bean Validation
`service`	Бизнес-логика: транзакции, проверка ссылок на вспомогательные объекты, каскадное удаление, вызов специальных операций
`service.exception`	Исключения бизнес-логики: «не найдено» и «некорректный ввод»
`repository`	Уровень хранения: репозитории Spring Data JPA (Hibernate) и вызов функций БД
`domain`	JPA-сущности Organization, Coordinates, Address и перечисление OrganizationType
`realtime`	События изменений и их рассылка всем клиентам по WebSocket после фиксации транзакции
`db.migration`	Миграции Flyway: таблицы и ограничения (V1), функции PL/pgSQL для специальных операций (V2)
`static`	Клиент: страницы и JavaScript-модули, общение с сервером через REST и WebSocket

2.2. Диаграммы классов

Доменная модель

Координаты и адреса хранятся в отдельных таблицах и связаны с организацией ассоциациями «многие к одному», поэтому новую организацию можно привязать к уже существующим координатам и адресам. Ограничения из задания показаны на диаграмме в фигурных скобках. Они продублированы аннотациями Bean Validation и JPA в сущностях и ограничениями `NOT NULL` и `CHECK` в таблицах БД. Идентификаторы генерирует PostgreSQL (`GENERATED ALWAYS AS IDENTITY`), дата создания заполняется перед сохранением в методе `onCreate()` (`@PrePersist`).

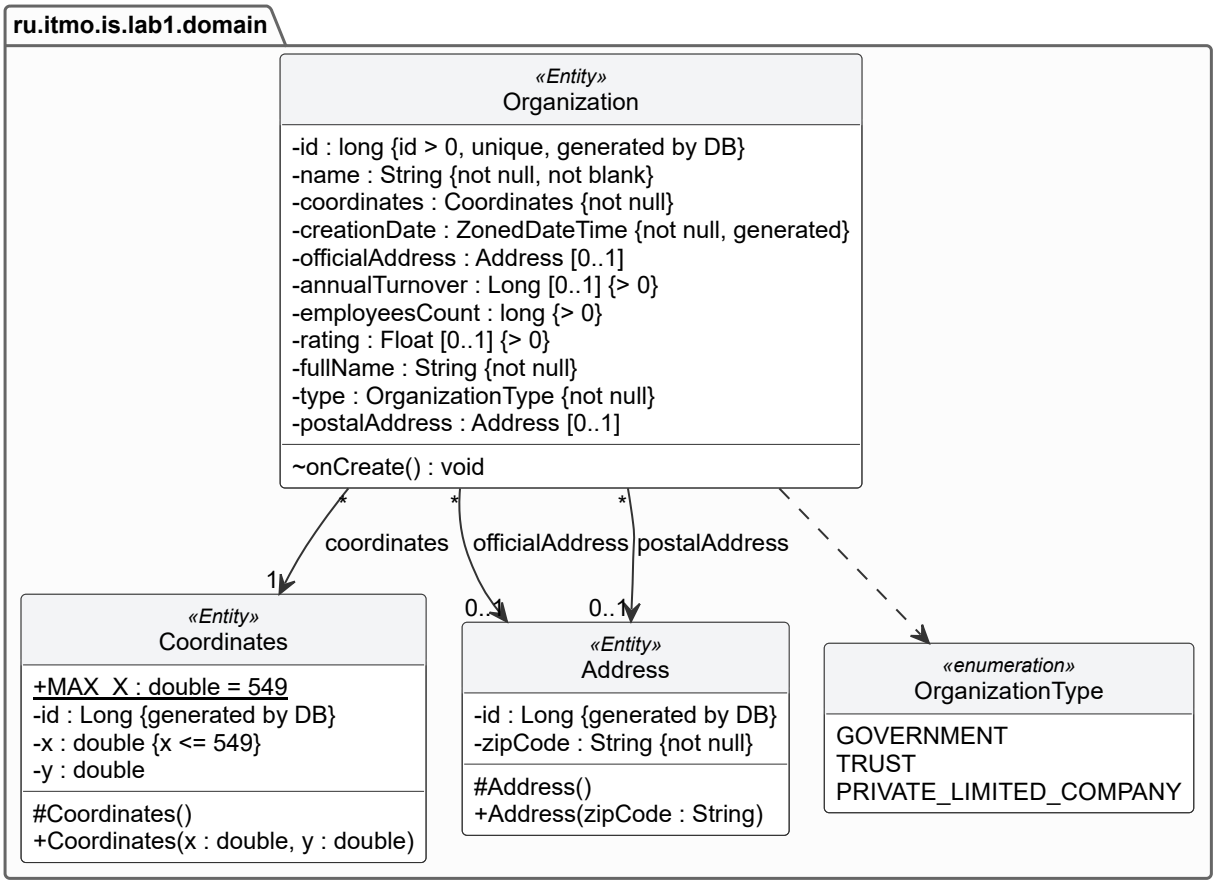


Рисунок 2 — Диаграмма классов доменной модели (пакет domain)

Объекты передачи данных

Связанный объект в запросе передаётся одним из двух способов: идентификатором уже существующего объекта (`id`) или значениями полей для создания нового. Для адресов значение `null` означает, что адреса нет. Поле `usages` показывает, сколько организаций ссылается на координаты или адрес; оно заполняется только в списках вспомогательных объектов.

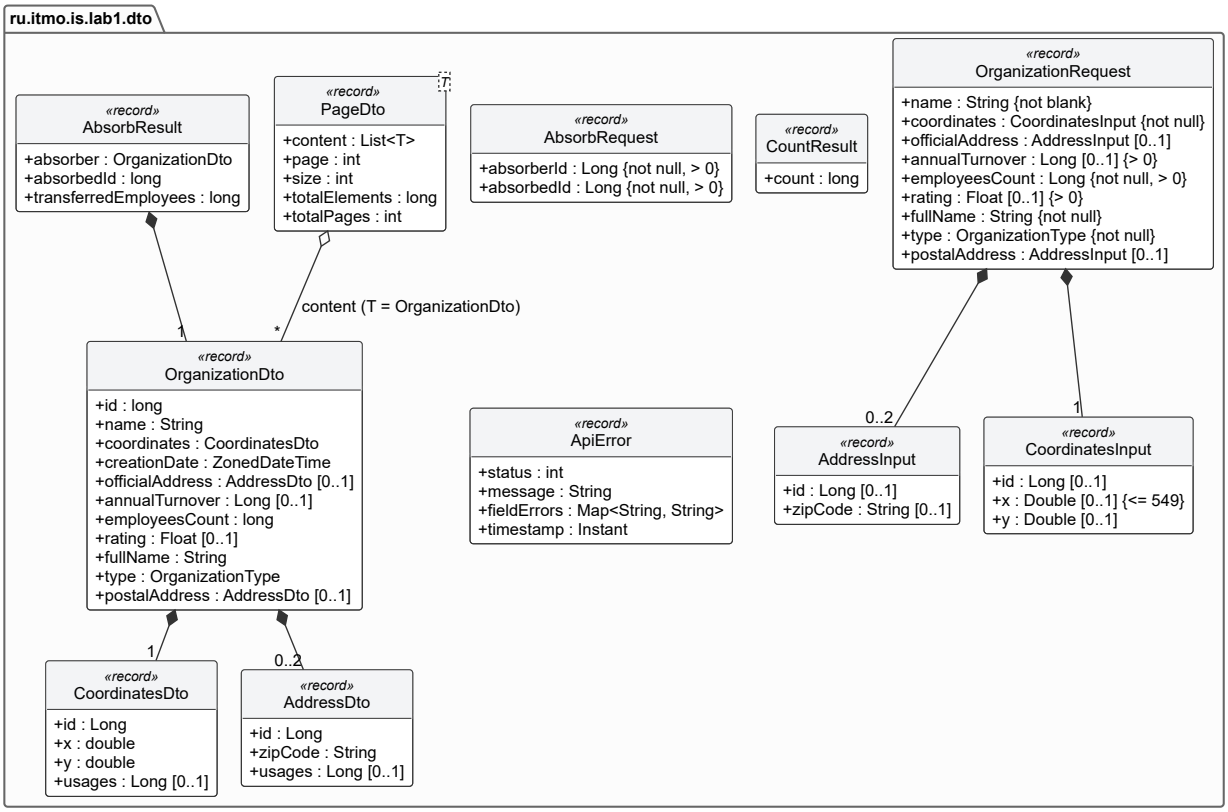


Рисунок 3 — Диаграмма классов объектов передачи данных (пакет dto)

Уровень представления

Контроллеры только принимают HTTP-запросы и передают их сервисам, все операции выполняются на сервере. `ApiExceptionHandler` переводит исключения (ошибки валидации, некорректный JSON, нарушение ограничений БД, ошибки из функций БД) в ответ с общим сообщением и сообщениями по каждому полю формы.

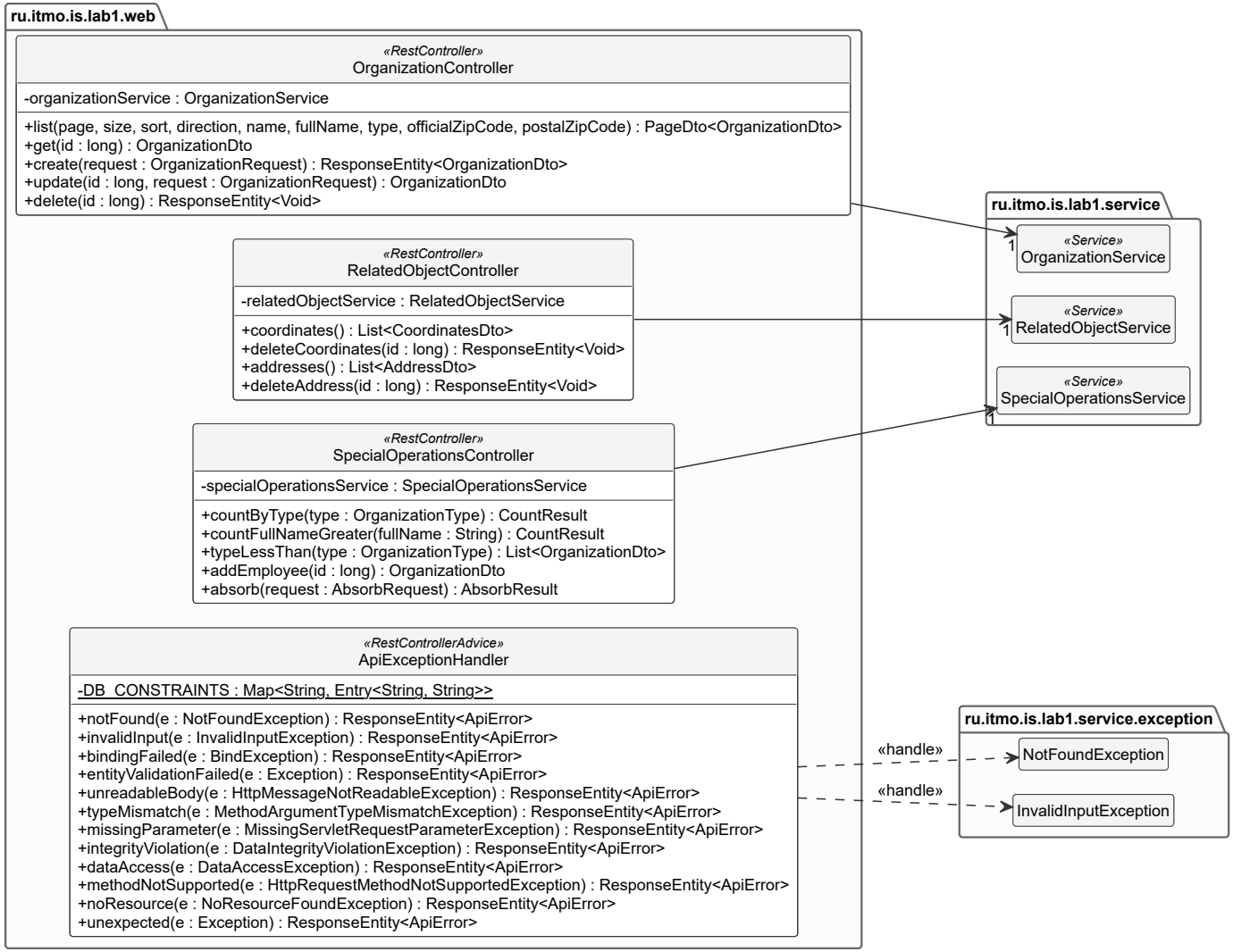


Рисунок 4 — Диаграмма классов уровня представления (пакет web)

Уровень бизнес-логики

Каждый публичный метод сервиса выполняется в транзакции. `OrganizationService` до сохранения проверяет все поля и ссылки на существующие координаты и адреса, поэтому новые вспомогательные объекты попадают в БД только при корректном запросе. `CascadeDeleter` собирает правила удаления в одном месте: вместе с организацией удаляются её координаты и адреса, если на них больше никто не ссылается.

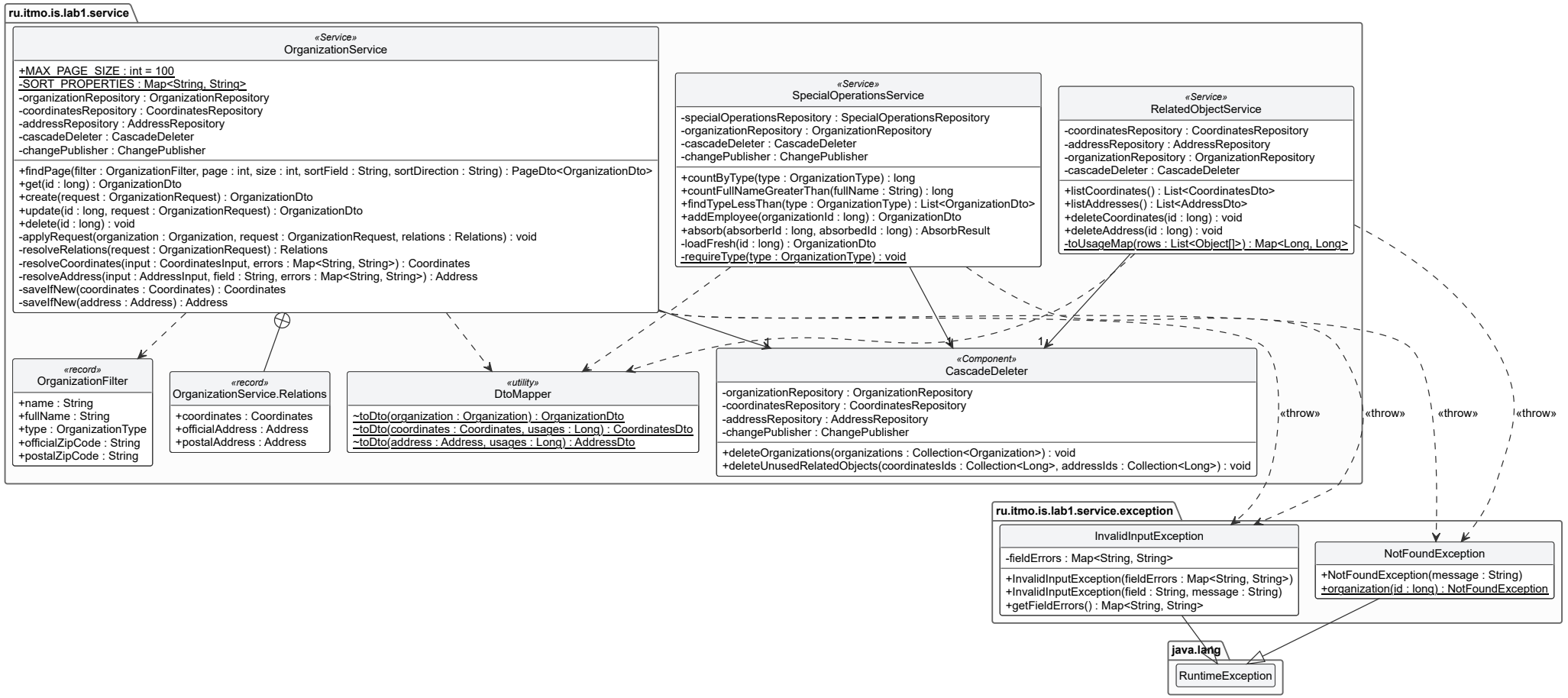


Рисунок 5 — Диаграмма классов уровня бизнес-логики (пакеты service и service.exception)

Уровень хранения

Репозитории Spring Data JPA работают через Hibernate. Фильтрация таблицы по полному совпадению строится через Criteria API (`OrganizationSpecifications`), а пагинация и сортировка выполняются запросом к БД. Специальные операции вызываются только из `SpecialOperationsRepository` нативными запросами к функциям PL/pgSQL.

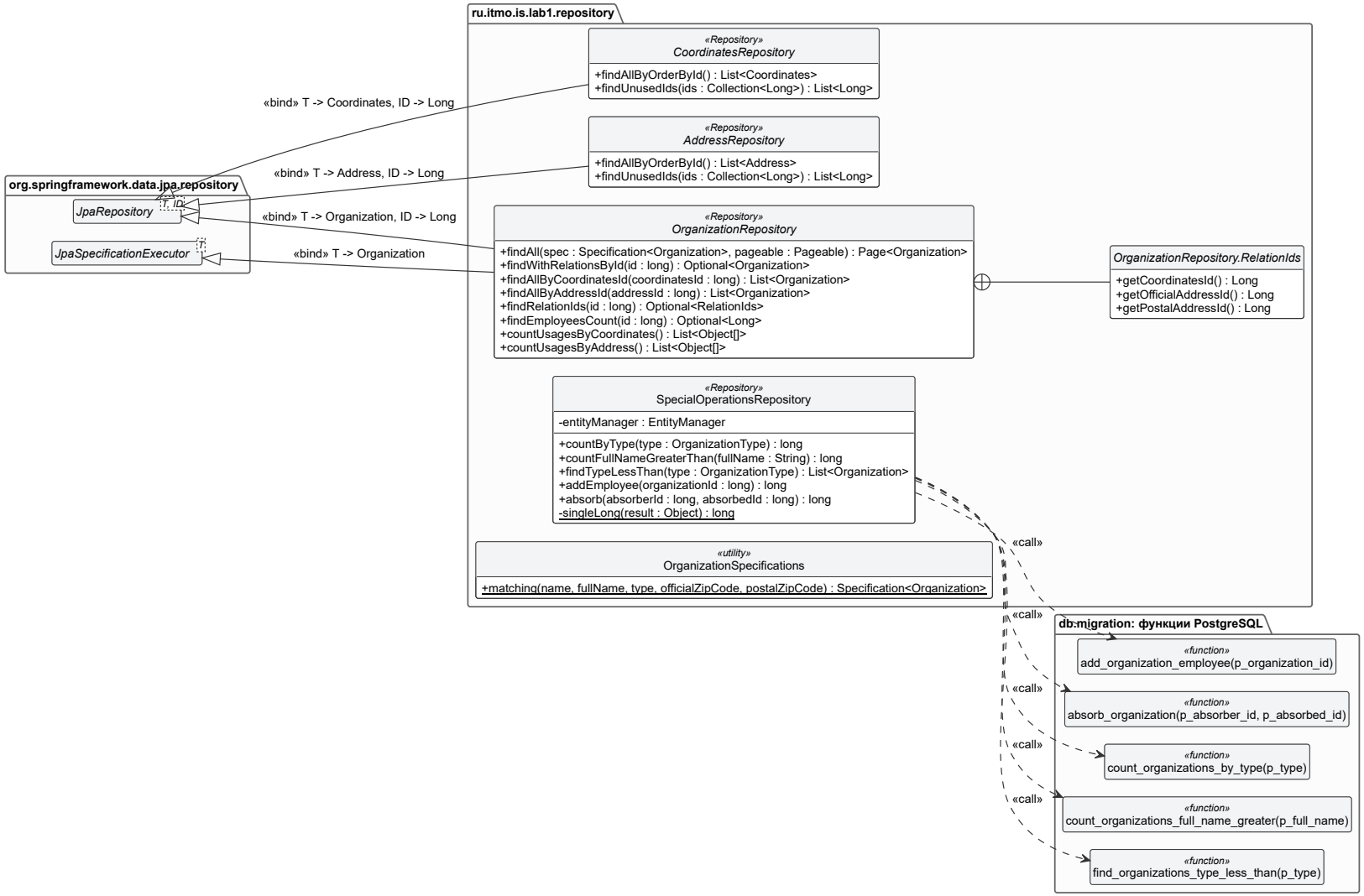


Рисунок 6 — Диаграмма классов уровня хранения (пакет repository) и функции БД

Оповещение клиентов об изменениях

Сервис публикует `EntityChangedEvent` внутри транзакции, а `ChangeBroadcaster` обрабатывает его с помощью `@TransactionalEventListener(phase = AFTER_COMMIT)`: сообщение уходит всем подключённым клиентам только после фиксации транзакции, а откатившиеся изменения не рассылаются. Получив событие, клиент заново запрашивает данные и обновляет таблицу, страницу объекта или открытое окно изменения.

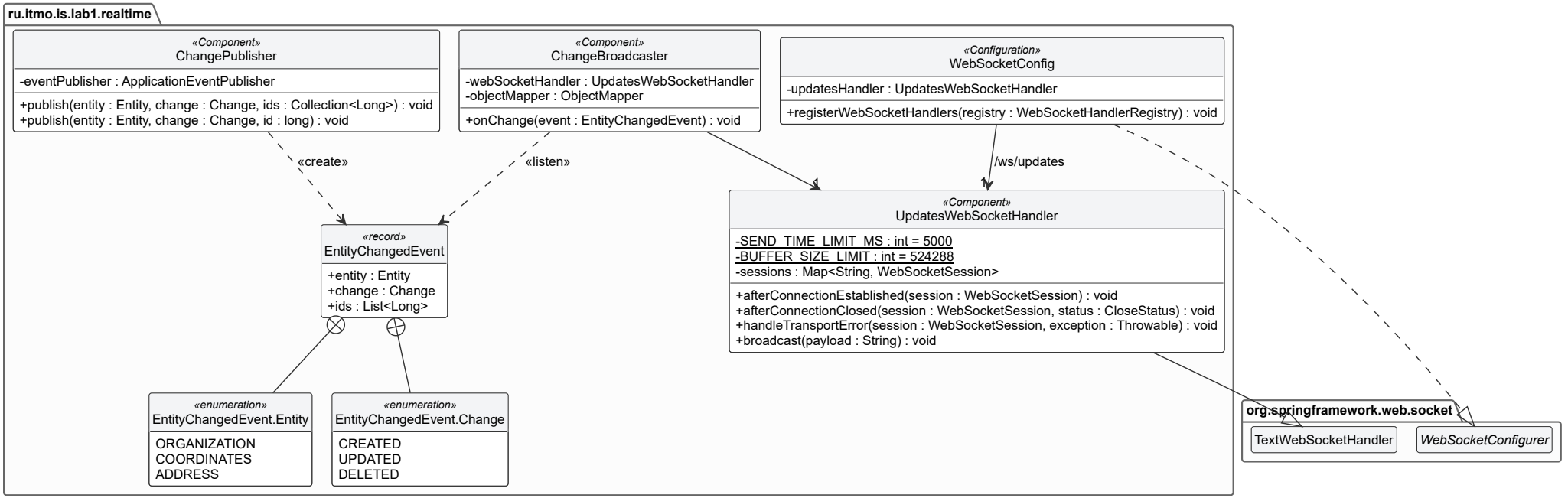


Рисунок 7 — Диаграмма классов пакета realtime

Клиентская часть

Каждой странице соответствует отдельный модуль из каталога `pages`. Общие части вынесены в модули: меню и уведомления (`layout.js`), форма организации с проверкой ввода (`OrganizationForm`), модальные окна изменения и подтверждения удаления (`dialogs.js`), REST-клиент (`api.js`) и подписка на изменения по WebSocket с автоматическим переподключением (`realtime.js`).

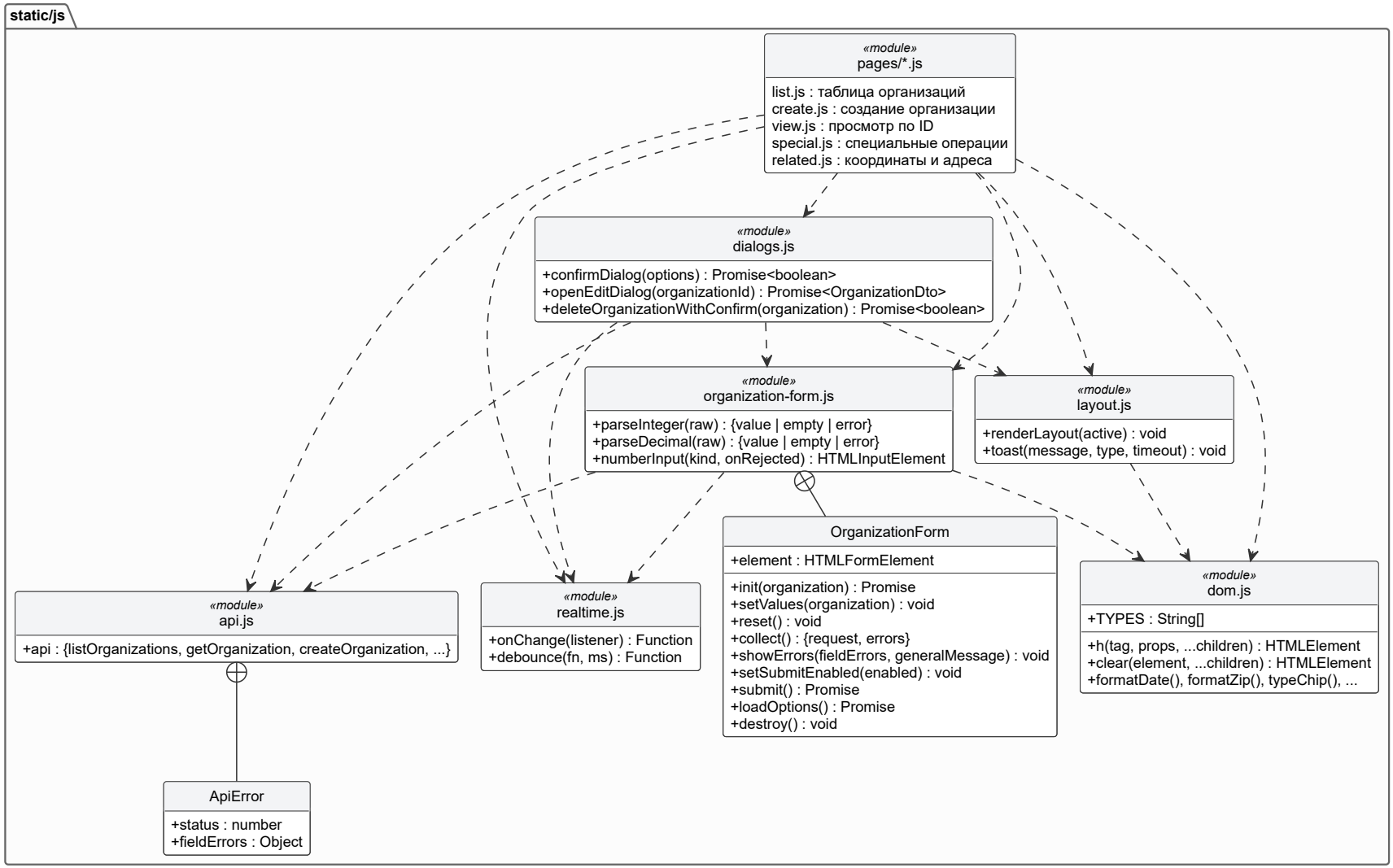


Рисунок 8 — Диаграмма модулей клиентской части

3. Исходный код

Ссылка на репозиторий с исходным кодом: _____

Полный листинг исходного кода приведён в Приложении А. Структура проекта:

```
is_1/
├── build.gradle, settings.gradle        сборка (Gradle, Spring Boot 3.5)
├── docs/uml/                            исходные описания UML-диаграмм (PlantUML)
└── src/main/
    ├── java/ru/itmo/is/lab1/
    │   ├── IsLab1Application.java       точка входа
    │   ├── domain/                      сущности JPA
    │   ├── dto/                         запросы и ответы REST API
    │   ├── repository/                  Spring Data JPA, вызов функций БД
    │   ├── service/                     бизнес-логика, каскадное удаление
    │   │   └── exception/               исключения бизнес-логики
    │   ├── realtime/                    WebSocket-оповещения
    │   └── web/                         REST-контроллеры, обработка ошибок
    └── resources/
        ├── application.yml              настройки (профиль helios: pg:5432/studs)
        ├── db/migration/                V1 — схема БД, V2 — функции PL/pgSQL
        └── static/                      HTML-страницы, CSS, JavaScript-модули
```

Таблица 2 — Соответствие требований задания и реализации

Требование	**Реализация**
Создание, просмотр по ИД, изменение, удаление в отдельных окнах	Страницы «Создать» и «Поиск по ID», модальное окно изменения, окно подтверждения удаления
Связь нового объекта с существующими вспомогательными объектами	Для координат и адресов в форме выбирается режим «Новые» или «Существующие» (список из БД)
Таблица, пагинация, сортировка, фильтрация по полному совпадению	`OrganizationService.findPage`, `OrganizationSpecifications`, страница `index.html`
Изменение из таблицы и из области визуализации объекта	Кнопка «Изменить» в строке таблицы и на странице просмотра объекта
Автоматическое обновление у других пользователей	WebSocket `/ws/updates`, рассылка после коммита транзакции
Удаление связанных объектов	`CascadeDeleter`, внешние ключи `ON DELETE CASCADE`
Специальные операции как функции БД	`V2__special_operations.sql`, вызов из `SpecialOperationsService`
Ограничения полей на уровне ORM и БД	Аннотации в `domain`, ограничения в `V1__schema.sql`
Генерация id средствами БД	`GENERATED ALWAYS AS IDENTITY`, `@GeneratedValue(strategy = IDENTITY)`
Реакция на некорректный ввод	Фильтрация недопустимых символов и проверки на клиенте, серверная валидация с сообщениями по полям
Навигация через меню	Общее меню на всех страницах (`layout.js`)

Сборка и запуск: `gradle bootJar`, затем `java -jar build/libs/is-lab1.jar --spring.profiles.active=helios` с переменными окружения `DB_USER`, `DB_PASSWORD` и `PORT`. Таблицы и функции БД создаются автоматически миграциями Flyway при первом запуске.

4. Выводы

В ходе лабораторной работы разработана информационная система для управления объектами класса Organization и связанными с ними объектами Coordinates и Address. Серверная часть построена на Spring MVC, уровень хранения — на Hibernate, данные хранятся в PostgreSQL. Приложение разделено на уровни представления, бизнес-логики и хранения; рассылка изменений вынесена в отдельный компонент.

Все операции над объектами выполняются на сервере в транзакциях. Координаты и адреса хранятся в отдельных таблицах, поэтому при создании организации её можно связать с уже существующими вспомогательными объектами. Требования к полям класса выполнены дважды: аннотациями Bean Validation и JPA на уровне ORM и ограничениями `NOT NULL` и `CHECK` на уровне БД. Идентификаторы генерирует PostgreSQL. Схему БД создают миграции Flyway, а Hibernate при запуске только сверяет с ней маппинг сущностей, поэтому расхождение кода и базы обнаруживается сразу.

Специальные операции реализованы функциями PL/pgSQL и вызываются из уровня бизнес-логики. Порядок значений перечисления для операции «type меньше заданного» совпадает с порядком объявления enum в Java. Значения fullName сравниваются побайтово (`COLLATE "C"`), чтобы результат не зависел от локали сервера БД. При поглощении обе организации блокируются (`SELECT … FOR UPDATE`) в фиксированном порядке, чтобы параллельные поглощения не приводили к взаимоблокировке.

Изменения доставляются всем клиентам по WebSocket. Событие рассылается только после фиксации транзакции, поэтому клиент, перезапросив данные, видит уже сохранённое состояние, а изменения из откатившихся транзакций до клиентов не доходят. Если объект изменили или удалили, пока у пользователя открыто окно его изменения, система предупреждает об этом.

В процессе работы пришлось учесть несколько неочевидных моментов:

- в PostgreSQL значение `NaN` считается больше любого числа, а `Infinity` проходит проверку «больше 0», поэтому ограничения для координат и рейтинга дополнены явными проверками на конечность;
- одни и те же координаты или адрес могут использоваться несколькими организациями, поэтому при удалении организации связанные объекты удаляются только тогда, когда на них больше никто не ссылается, а удаление

самих координат или адреса каскадно удаляет все ссылающиеся на них организации;

- по той же причине изменение координат или адреса в форме создаёт новый объект, а не меняет общий, иначе правка одной организации незаметно затронула бы другие.

В результате выполнения работы получен практический опыт проектирования многоуровневого веб-приложения на Spring, отображения объектной модели на реляционную с помощью Hibernate, переноса бизнес-операций в хранимые функции СУБД и синхронизации состояния между несколькими клиентами в реальном времени.

Приложение А. Листинг исходного кода

Листинг А.1 — build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.6'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'ru.itmo.is'
version = '1.0.0'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-websocket'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.flywaydb:flyway-core'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.postgresql:postgresql'

    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.withType(JavaCompile).configureEach {
    options.encoding = 'UTF-8'
    options.compilerArgs << '-parameters'
}

tasks.named('test') {
    useJUnitPlatform()
}

bootJar {
    archiveFileName = 'is-lab1.jar'
}

jar {
    enabled = false
}
```

Листинг А.2 — settings.gradle

```
pluginManagement {
    repositories {
        // Зеркало Maven Central: repo.maven.apache.org из РФ отвечает 403
        maven { url 'https://maven-central.storage-download.googleapis.com/maven2/' }
        gradlePluginPortal()
        mavenCentral()
    }
}

dependencyResolutionManagement {
    repositories {
        maven { url 'https://maven-central.storage-download.googleapis.com/maven2/' }
        mavenCentral()
    }
}

rootProject.name = 'is-lab1'
```

Листинг А.3 — src/main/resources/application.yml

```
server:
  port: ${PORT:8080}
  servlet:
    encoding:
      charset: UTF-8
```

```
      force: true

spring:
  application:
    name: is-lab1
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/is_lab1}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:123}
  jpa:
    open-in-view: false
    hibernate:
      # Схему создаёт Flyway, Hibernate только сверяет маппинг с таблицами
      ddl-auto: validate
    properties:
      hibernate:
        default_batch_fetch_size: 50
        jdbc:
          time_zone: UTC
  flyway:
    enabled: true
    locations: classpath:db/migration
    # На кафедральном pg в схеме студента могут лежать таблицы других лаб
    baseline-on-migrate: true
    baseline-version: 0
  jackson:
    deserialization:
      accept-float-as-int: false
      fail-on-null-for-primitives: true
    default-property-inclusion: always

logging:
  level:
    org.hibernate.SQL: ${SQL_LOG_LEVEL:warn}

---
# Запуск на гелиосе: java -jar is-lab1.jar --spring.profiles.active=helios
spring:
  config:
    activate:
      on-profile: helios
  datasource:
    url: ${DB_URL:jdbc:postgresql://pg:5432/studs}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
```

Листинг A.4 — src/main/resources/db/migration/V1__schema.sql

```
-- Вспомогательные объекты хранятся в отдельных таблицах, чтобы новую организацию
-- можно было связать с уже существующими координатами/адресами.

CREATE TABLE coordinates
(
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    x  DOUBLE PRECISION NOT NULL,
    y  DOUBLE PRECISION NOT NULL,
    CONSTRAINT coordinates_id_positive CHECK (id > 0),
    -- NaN и +Infinity в PostgreSQL больше любого числа, поэтому тоже отсекаются
    CONSTRAINT coordinates_x_max CHECK (x <= 549),
    CONSTRAINT coordinates_x_finite CHECK (x <> '-Infinity'::DOUBLE PRECISION),
    CONSTRAINT coordinates_y_finite CHECK (y NOT IN ('NaN'::DOUBLE PRECISION,
                                                     'Infinity'::DOUBLE PRECISION,
                                                     '-Infinity'::DOUBLE PRECISION))
);

CREATE TABLE address
(
    id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zip_code VARCHAR(255) NOT NULL,
    CONSTRAINT address_id_positive CHECK (id > 0)
);

CREATE TABLE organization
(
    id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name                VARCHAR(255)             NOT NULL,
    coordinates_id      BIGINT                   NOT NULL
        CONSTRAINT organization_coordinates_fk REFERENCES coordinates (id) ON DELETE CASCADE,
```

```
    creation_date       TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    official_address_id BIGINT
        CONSTRAINT organization_official_address_fk REFERENCES address (id) ON DELETE CASCADE,
    annual_turnover     BIGINT,
    employees_count     BIGINT                   NOT NULL,
    rating              REAL,
    full_name           VARCHAR(255)             NOT NULL,
    type                VARCHAR(32)              NOT NULL,
    postal_address_id   BIGINT
        CONSTRAINT organization_postal_address_fk REFERENCES address (id) ON DELETE CASCADE,
    CONSTRAINT organization_id_positive CHECK (id > 0),
    CONSTRAINT organization_name_not_empty CHECK (btrim(name) <> ''),
    CONSTRAINT organization_annual_turnover_positive CHECK (annual_turnover > 0),
    CONSTRAINT organization_employees_count_positive CHECK (employees_count > 0),
    CONSTRAINT organization_rating_positive CHECK (rating > 0 AND rating <> 'NaN'::REAL AND rating <>
'Infinity'::REAL),
    CONSTRAINT organization_type_valid CHECK (type IN ('GOVERNMENT', 'TRUST', 'PRIVATE_LIMITED_COMPANY'))
);

CREATE INDEX organization_coordinates_idx ON organization (coordinates_id);
CREATE INDEX organization_official_address_idx ON organization (official_address_id);
CREATE INDEX organization_postal_address_idx ON organization (postal_address_id);
CREATE INDEX organization_name_idx ON organization (name);
CREATE INDEX organization_full_name_idx ON organization (full_name);
CREATE INDEX organization_type_idx ON organization (type);
```

Листинг A.5 — src/main/resources/db/migration/V2__special_operations.sql

```
-- Специальные операции над организациями. Вызываются из слоя бизнес-логики
-- (ru.itmo.is.lab1.repository.SpecialOperationsRepository).

-- Порядковый номер типа в порядке объявления Java-enum OrganizationType.
CREATE OR REPLACE FUNCTION organization_type_rank(p_type VARCHAR)
    RETURNS INTEGER
    LANGUAGE sql
    IMMUTABLE
AS
$$
SELECT array_position(ARRAY ['GOVERNMENT', 'TRUST', 'PRIVATE_LIMITED_COMPANY']::VARCHAR[], p_type)
$$;

-- 1. Количество организаций, у которых type равен заданному.
CREATE OR REPLACE FUNCTION count_organizations_by_type(p_type VARCHAR)
    RETURNS BIGINT
    LANGUAGE plpgsql
    STABLE
AS
$$
BEGIN
    IF organization_type_rank(p_type) IS NULL THEN
        RAISE EXCEPTION 'Неизвестный тип организации: %', p_type USING ERRCODE = '22023';
    END IF;
    RETURN (SELECT count(*) FROM organization WHERE type = p_type);
END;
$$;

-- 2. Количество организаций, у которых fullName больше заданного (сравнение строк).
CREATE OR REPLACE FUNCTION count_organizations_full_name_greater(p_full_name VARCHAR)
    RETURNS BIGINT
    LANGUAGE plpgsql
    STABLE
AS
$$
BEGIN
    IF p_full_name IS NULL THEN
        RAISE EXCEPTION 'Значение fullName для сравнения не задано' USING ERRCODE = '22004';
    END IF;
    RETURN (SELECT count(*) FROM organization WHERE full_name > p_full_name);
END;
$$;

-- 3. Организации, у которых type меньше заданного (по порядку объявления enum).
CREATE OR REPLACE FUNCTION find_organizations_type_less_than(p_type VARCHAR)
    RETURNS SETOF organization
    LANGUAGE plpgsql
    STABLE
AS
$$
```

```
DECLARE
    v_rank INTEGER := organization_type_rank(p_type);
BEGIN
    IF v_rank IS NULL THEN
        RAISE EXCEPTION 'Неизвестный тип организации: %', p_type USING ERRCODE = '22023';
    END IF;
    RETURN QUERY
        SELECT o.*
        FROM organization o
        WHERE organization_type_rank(o.type) < v_rank
        ORDER BY o.id;
END;
$$;

-- 4. Добавить сотрудника в организацию. Возвращает новое число сотрудников.
CREATE OR REPLACE FUNCTION add_organization_employee(p_organization_id BIGINT)
    RETURNS BIGINT
    LANGUAGE plpgsql
AS
$$
DECLARE
    v_count BIGINT;
BEGIN
    UPDATE organization
    SET employees_count = employees_count + 1
    WHERE id = p_organization_id
    RETURNING employees_count INTO v_count;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Организация с id = % не найдена', p_organization_id USING ERRCODE = 'P0002';
    END IF;
    RETURN v_count;
END;
$$;

-- 5. Поглощение: все сотрудники (и годовой оборот) поглощаемой организации переходят
--    к поглощающей, поглощаемая организация удаляется. Возвращает новое число сотрудников.
CREATE OR REPLACE FUNCTION absorb_organization(p_absorber_id BIGINT, p_absorbed_id BIGINT)
    RETURNS BIGINT
    LANGUAGE plpgsql
AS
$$
DECLARE
    v_absorbed organization%ROWTYPE;
    v_count    BIGINT;
BEGIN
    IF p_absorber_id = p_absorbed_id THEN
        RAISE EXCEPTION 'Организация не может поглотить саму себя' USING ERRCODE = '22023';
    END IF;

    -- Блокируем обе строки в одном порядке, чтобы параллельные поглощения не взаимоблокировались
    PERFORM 1 FROM organization WHERE id IN (p_absorber_id, p_absorbed_id) ORDER BY id FOR UPDATE;

    SELECT * INTO v_absorbed FROM organization WHERE id = p_absorbed_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Поглощаемая организация с id = % не найдена', p_absorbed_id USING ERRCODE = 'P0002';
    END IF;

    UPDATE organization
    SET employees_count = employees_count + v_absorbed.employees_count,
        annual_turnover = CASE
                              WHEN annual_turnover IS NULL AND v_absorbed.annual_turnover IS NULL THEN NULL
                              ELSE coalesce(annual_turnover, 0) + coalesce(v_absorbed.annual_turnover, 0)
            END
    WHERE id = p_absorber_id
    RETURNING employees_count INTO v_count;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Поглощающая организация с id = % не найдена', p_absorber_id USING ERRCODE = 'P0002';
    END IF;

    DELETE FROM organization WHERE id = p_absorbed_id;
    RETURN v_count;
END;
$$;
```

Листинг A.6 — src/main/resources/db/migration/V3__full_name_byte_order.sql

```
-- «fullName больше заданного»: строки сравниваются побайтово (COLLATE "C"), а не по правилам локали БД.
-- На studs локаль ru_RU.UTF-8, в ней регистр и пробелы при сравнении почти не учитываются,
-- и результат зависел бы от настроек сервера. Порядок "C" для UTF-8 совпадает с порядком
-- кодовых точек Unicode, то есть с String.compareTo в Java (для символов из BMP).
CREATE OR REPLACE FUNCTION count_organizations_full_name_greater(p_full_name VARCHAR)
    RETURNS BIGINT
    LANGUAGE plpgsql
    STABLE
AS
$$
BEGIN
    IF p_full_name IS NULL THEN
        RAISE EXCEPTION 'Значение fullName для сравнения не задано' USING ERRCODE = '22004';
    END IF;
    RETURN (SELECT count(*) FROM organization WHERE full_name COLLATE "C" > p_full_name COLLATE "C");
END;
$$;
```

Листинг A.7 — src/main/java/ru/itmo/is/lab1/IsLab1Application.java

```
package ru.itmo.is.lab1;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class IsLab1Application {

    public static void main(String[] args) {
        SpringApplication.run(IsLab1Application.class, args);
    }
}
```

Листинг A.8 — src/main/java/ru/itmo/is/lab1/domain/Address.java

```
package ru.itmo.is.lab1.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

@Entity
@Table(name = "address")
public class Address {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotNull(message = "Почтовый индекс не может быть null")
    @Size(max = 255, message = "Почтовый индекс не длиннее 255 символов")
    @Column(name = "zip_code", nullable = false, length = 255)
    private String zipCode;

    protected Address() {
    }

    public Address(String zipCode) {
        this.zipCode = zipCode;
    }

    public Long getId() {
        return id;
    }

    public String getZipCode() {
        return zipCode;
    }
}
```

Листинг A.9 — src/main/java/ru/itmo/is/lab1/domain/Coordinates.java

```
package ru.itmo.is.lab1.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Max;

@Entity
@Table(name = "coordinates")
public class Coordinates {

    public static final double MAX_X = 549;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Max(value = (long) MAX_X, message = "X не может быть больше 549")
    @Column(name = "x", nullable = false)
    private double x;

    @Column(name = "y", nullable = false)
    private double y;

    protected Coordinates() {
    }

    public Coordinates(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Long getId() {
        return id;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }
}
```

Листинг A.10 — src/main/java/ru/itmo/is/lab1/domain/Organization.java

```
package ru.itmo.is.lab1.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.PrePersist;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

import java.time.ZonedDateTime;

@Entity
@Table(name = "organization")
public class Organization {

    // Генерируется БД (identity); положительность гарантирует CHECK (id > 0)
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private long id;

@NotBlank(message = "Название не может быть пустым")
@Size(max = 255, message = "Название не длиннее 255 символов")
@Column(name = "name", nullable = false, length = 255)
private String name;

@NotNull(message = "Координаты обязательны")
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "coordinates_id", nullable = false)
private Coordinates coordinates;

@NotNull
@Column(name = "creation_date", nullable = false, updatable = false)
private ZonedDateTime creationDate;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "official_address_id")
private Address officialAddress;

@Positive(message = "Годовой оборот должен быть больше 0")
@Column(name = "annual_turnover")
private Long annualTurnover;

@Positive(message = "Количество сотрудников должно быть больше 0")
@Column(name = "employees_count", nullable = false)
private long employeesCount;

@Positive(message = "Рейтинг должен быть больше 0")
@Column(name = "rating")
private Float rating;

@NotNull(message = "Полное название не может быть null")
@Size(max = 255, message = "Полное название не длиннее 255 символов")
@Column(name = "full_name", nullable = false, length = 255)
private String fullName;

@NotNull(message = "Тип организации обязателен")
@Enumerated(EnumType.STRING)
@Column(name = "type", nullable = false, length = 32)
private OrganizationType type;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "postal_address_id")
private Address postalAddress;

@PrePersist
void onCreate() {
    if (creationDate == null) {
        creationDate = ZonedDateTime.now();
    }
}

public long getId() {
    return id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public Coordinates getCoordinates() {
    return coordinates;
}

public void setCoordinates(Coordinates coordinates) {
    this.coordinates = coordinates;
}

public ZonedDateTime getCreationDate() {
    return creationDate;
}
```

```
    public Address getOfficialAddress() {
        return officialAddress;
    }

    public void setOfficialAddress(Address officialAddress) {
        this.officialAddress = officialAddress;
    }

    public Long getAnnualTurnover() {
        return annualTurnover;
    }

    public void setAnnualTurnover(Long annualTurnover) {
        this.annualTurnover = annualTurnover;
    }

    public long getEmployeesCount() {
        return employeesCount;
    }

    public void setEmployeesCount(long employeesCount) {
        this.employeesCount = employeesCount;
    }

    public Float getRating() {
        return rating;
    }

    public void setRating(Float rating) {
        this.rating = rating;
    }

    public String getFullName() {
        return fullName;
    }

    public void setFullName(String fullName) {
        this.fullName = fullName;
    }

    public OrganizationType getType() {
        return type;
    }

    public void setType(OrganizationType type) {
        this.type = type;
    }

    public Address getPostalAddress() {
        return postalAddress;
    }

    public void setPostalAddress(Address postalAddress) {
        this.postalAddress = postalAddress;
    }
}
```

Листинг A.11 — src/main/java/ru/itmo/is/lab1/domain/OrganizationType.java

```
package ru.itmo.is.lab1.domain;

/**
 * Порядок констант важен: операция "type меньше заданного" сравнивает типы
 * по порядку объявления (см. функцию БД organization_type_rank).
 */
public enum OrganizationType {
    GOVERNMENT,
    TRUST,
    PRIVATE_LIMITED_COMPANY
}
```

Листинг A.12 — src/main/java/ru/itmo/is/lab1/dto/AbsorbRequest.java

```
package ru.itmo.is.lab1.dto;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
```

```
public record AbsorbRequest(
        @NotNull(message = "Укажите id поглощающей организации")
        @Positive(message = "id должен быть больше 0")
        Long absorberId,

        @NotNull(message = "Укажите id поглощаемой организации")
        @Positive(message = "id должен быть больше 0")
        Long absorbedId
) {
}
```

Листинг A.13 — src/main/java/ru/itmo/is/lab1/dto/AbsorbResult.java

```
package ru.itmo.is.lab1.dto;

public record AbsorbResult(OrganizationDto absorber, long absorbedId, long transferredEmployees) {
}
```

Листинг A.14 — src/main/java/ru/itmo/is/lab1/dto/AddressDto.java

```
package ru.itmo.is.lab1.dto;

import com.fasterxml.jackson.annotation.JsonInclude;

/**
 * @param usages сколько организаций ссылается на этот адрес (не передаётся во вложенных объектах)
 */
public record AddressDto(Long id, String zipCode,
                         @JsonInclude(JsonInclude.Include.NON_NULL) Long usages) {
}
```

Листинг A.15 — src/main/java/ru/itmo/is/lab1/dto/AddressInput.java

```
package ru.itmo.is.lab1.dto;

import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

/**
 * Либо {@code id} уже существующего адреса, либо {@code zipCode} для нового.
 * Отсутствие объекта целиком (null) означает, что адреса нет.
 */
public record AddressInput(
        @Positive(message = "Некорректный идентификатор адреса")
        Long id,

        @Size(max = 255, message = "Почтовый индекс не длиннее 255 символов")
        String zipCode
) {
}
```

Листинг A.16 — src/main/java/ru/itmo/is/lab1/dto/ApiError.java

```
package ru.itmo.is.lab1.dto;

import java.time.Instant;
import java.util.Map;

/**
 * @param fieldErrors сообщения по полям формы; ключ — путь к полю, например {@code coordinates.x}
 */
public record ApiError(int status, String message, Map<String, String> fieldErrors, Instant timestamp) {

    public ApiError(int status, String message, Map<String, String> fieldErrors) {
        this(status, message, fieldErrors, Instant.now());
    }

    public ApiError(int status, String message) {
        this(status, message, Map.of());
    }
}
```

Листинг A.17 — src/main/java/ru/itmo/is/lab1/dto/CoordinatesDto.java

```
package ru.itmo.is.lab1.dto;
```

```
import com.fasterxml.jackson.annotation.JsonInclude;

/**
 * @param usages сколько организаций ссылается на эти координаты (не передаётся во вложенных объектах)
 */
public record CoordinatesDto(Long id, double x, double y,
                             @JsonInclude(JsonInclude.Include.NON_NULL) Long usages) {
}
```

Листинг A.18 — src/main/java/ru/itmo/is/lab1/dto/CoordinatesInput.java

```
package ru.itmo.is.lab1.dto;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Positive;

/**
 * Либо {@code id} уже существующих координат, либо значения {@code x}/{@code y} для новых.
 */
public record CoordinatesInput(
        @Positive(message = "Некорректный идентификатор координат")
        Long id,

        @Max(value = 549, message = "X не может быть больше 549")
        Double x,

        Double y
) {
}
```

Листинг A.19 — src/main/java/ru/itmo/is/lab1/dto/CountResult.java

```
package ru.itmo.is.lab1.dto;

public record CountResult(long count) {
}
```

Листинг A.20 — src/main/java/ru/itmo/is/lab1/dto/OrganizationDto.java

```
package ru.itmo.is.lab1.dto;

import ru.itmo.is.lab1.domain.OrganizationType;

import java.time.ZonedDateTime;

public record OrganizationDto(
        long id,
        String name,
        CoordinatesDto coordinates,
        ZonedDateTime creationDate,
        AddressDto officialAddress,
        Long annualTurnover,
        long employeesCount,
        Float rating,
        String fullName,
        OrganizationType type,
        AddressDto postalAddress
) {
}
```

Листинг A.21 — src/main/java/ru/itmo/is/lab1/dto/OrganizationRequest.java

```
package ru.itmo.is.lab1.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;
import ru.itmo.is.lab1.domain.OrganizationType;

/** Тело запроса на создание и изменение организации. */
public record OrganizationRequest(
        @NotBlank(message = "Название не может быть пустым")
        @Size(max = 255, message = "Название не длиннее 255 символов")
        String name,
```

```
        @NotNull(message = "Координаты обязательны")
        @Valid
        CoordinatesInput coordinates,

        @Valid
        AddressInput officialAddress,

        @Positive(message = "Годовой оборот должен быть больше 0")
        Long annualTurnover,

        @NotNull(message = "Укажите количество сотрудников")
        @Positive(message = "Количество сотрудников должно быть больше 0")
        Long employeesCount,

        @Positive(message = "Рейтинг должен быть больше 0")
        Float rating,

        @NotNull(message = "Полное название не может быть null")
        @Size(max = 255, message = "Полное название не длиннее 255 символов")
        String fullName,

        @NotNull(message = "Выберите тип организации")
        OrganizationType type,

        @Valid
        AddressInput postalAddress
) {
}
```

Листинг A.22 — src/main/java/ru/itmo/is/lab1/dto/PageDto.java

```
package ru.itmo.is.lab1.dto;

import java.util.List;

public record PageDto<T>(List<T> content, int page, int size, long totalElements, int totalPages) {
}
```

Листинг A.23 — src/main/java/ru/itmo/is/lab1/repository/AddressRepository.java

```
package ru.itmo.is.lab1.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import ru.itmo.is.lab1.domain.Address;

import java.util.Collection;
import java.util.List;

public interface AddressRepository extends JpaRepository<Address, Long> {

    List<Address> findAllByOrderById();

    /** Из переданных адресов оставляет те, на которые больше не ссылается ни одна организация. */
    @Query("""
            select a.id from Address a
            where a.id in :ids
              and not exists (
                  select o.id from Organization o
                  where o.officialAddress = a or o.postalAddress = a)
            """)
    List<Long> findUnusedIds(@Param("ids") Collection<Long> ids);
}
```

Листинг A.24 — src/main/java/ru/itmo/is/lab1/repository/CoordinatesRepository.java

```
package ru.itmo.is.lab1.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import ru.itmo.is.lab1.domain.Coordinates;

import java.util.Collection;
import java.util.List;
```

```
public interface CoordinatesRepository extends JpaRepository<Coordinates, Long> {

    List<Coordinates> findAllByOrderById();

    /** Из переданных координат оставляет те, на которые больше не ссылается ни одна организация. */
    @Query("""
            select c.id from Coordinates c
            where c.id in :ids
              and not exists (select o.id from Organization o where o.coordinates = c)
            """)
    List<Long> findUnusedIds(@Param("ids") Collection<Long> ids);
}
```

Листинг A.25 — src/main/java/ru/itmo/is/lab1/repository/OrganizationRepository.java

```
package ru.itmo.is.lab1.repository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import ru.itmo.is.lab1.domain.Organization;

import java.util.List;
import java.util.Optional;

public interface OrganizationRepository extends JpaRepository<Organization, Long>,
JpaSpecificationExecutor<Organization> {

    @Override
    @EntityGraph(attributePaths = {"coordinates", "officialAddress", "postalAddress"})
    Page<Organization> findAll(Specification<Organization> spec, Pageable pageable);

    @EntityGraph(attributePaths = {"coordinates", "officialAddress", "postalAddress"})
    Optional<Organization> findWithRelationsById(long id);

    @Query("select o from Organization o where o.coordinates.id = :coordinatesId")
    List<Organization> findAllByCoordinatesId(@Param("coordinatesId") long coordinatesId);

    @Query("""
            select o from Organization o
            where o.officialAddress.id = :addressId or o.postalAddress.id = :addressId
            """)
    List<Organization> findAllByAddressId(@Param("addressId") long addressId);

    /** Идентификаторы связанных объектов без загрузки самой организации. */
    @Query("""
            select c.id as coordinatesId, oa.id as officialAddressId, pa.id as postalAddressId
            from Organization o
                join o.coordinates c
                left join o.officialAddress oa
                left join o.postalAddress pa
            where o.id = :id
            """)
    Optional<RelationIds> findRelationIds(@Param("id") long id);

    @Query("select o.employeesCount from Organization o where o.id = :id")
    Optional<Long> findEmployeesCount(@Param("id") long id);

    @Query("select o.coordinates.id, count(o) from Organization o group by o.coordinates.id")
    List<Object[]> countUsagesByCoordinates();

    @Query("""
            select a.id, count(o) from Address a, Organization o
            where o.officialAddress = a or o.postalAddress = a
            group by a.id
            """)
    List<Object[]> countUsagesByAddress();

    interface RelationIds {
        Long getCoordinatesId();
```

```
        Long getOfficialAddressId();

        Long getPostalAddressId();
    }
}
```

Листинг A.26 — src/main/java/ru/itmo/is/lab1/repository/OrganizationSpecifications.java

```
package ru.itmo.is.lab1.repository;

import jakarta.persistence.criteria.JoinType;
import jakarta.persistence.criteria.Predicate;
import org.springframework.data.jpa.domain.Specification;
import ru.itmo.is.lab1.domain.Organization;
import ru.itmo.is.lab1.domain.OrganizationType;

import java.util.ArrayList;
import java.util.List;

/** Фильтры таблицы организаций: только полное совпадение значения. */
public final class OrganizationSpecifications {

    private OrganizationSpecifications() {
    }

    public static Specification<Organization> matching(String name,
                                                       String fullName,
                                                       OrganizationType type,
                                                       String officialZipCode,
                                                       String postalZipCode) {
        return (root, query, cb) -> {
            List<Predicate> predicates = new ArrayList<>();
            if (name != null) {
                predicates.add(cb.equal(root.get("name"), name));
            }
            if (fullName != null) {
                predicates.add(cb.equal(root.get("fullName"), fullName));
            }
            if (type != null) {
                predicates.add(cb.equal(root.get("type"), type));
            }
            if (officialZipCode != null) {
                predicates.add(cb.equal(root.join("officialAddress", JoinType.INNER).get("zipCode"),
officialZipCode));
            }
            if (postalZipCode != null) {
                predicates.add(cb.equal(root.join("postalAddress", JoinType.INNER).get("zipCode"),
postalZipCode));
            }
            return cb.and(predicates.toArray(Predicate[]::new));
        };
    }
}
```

Листинг A.27 — src/main/java/ru/itmo/is/lab1/repository/SpecialOperationsRepository.java

```
package ru.itmo.is.lab1.repository;

import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;
import ru.itmo.is.lab1.domain.Organization;
import ru.itmo.is.lab1.domain.OrganizationType;

import java.util.List;

/** Вызовы функций БД из V2__special_operations.sql. */
@Repository
public class SpecialOperationsRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public long countByType(OrganizationType type) {
        return singleLong(entityManager
```

```
                .createNativeQuery("select count_organizations_by_type(cast(:type as varchar))")
                .setParameter("type", type.name())
                .getSingleResult());
    }

    public long countFullNameGreaterThan(String fullName) {
        return singleLong(entityManager
                .createNativeQuery("select count_organizations_full_name_greater(cast(:fullName as varchar))")
                .setParameter("fullName", fullName)
                .getSingleResult());
    }

    @SuppressWarnings("unchecked")
    public List<Organization> findTypeLessThan(OrganizationType type) {
        return entityManager
                .createNativeQuery("select * from find_organizations_type_less_than(cast(:type as varchar))",
                        Organization.class)
                .setParameter("type", type.name())
                .getResultList();
    }

    public long addEmployee(long organizationId) {
        return singleLong(entityManager
                .createNativeQuery("select add_organization_employee(cast(:id as bigint))")
                .setParameter("id", organizationId)
                .getSingleResult());
    }

    public long absorb(long absorberId, long absorbedId) {
        return singleLong(entityManager
                .createNativeQuery("select absorb_organization(cast(:absorber as bigint), cast(:absorbed as
bigint))")
                .setParameter("absorber", absorberId)
                .setParameter("absorbed", absorbedId)
                .getSingleResult());
    }

    private static long singleLong(Object result) {
        return ((Number) result).longValue();
    }
}
```

Листинг A.28 — src/main/java/ru/itmo/is/lab1/service/CascadeDeleter.java

```
package ru.itmo.is.lab1.service;

import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import ru.itmo.is.lab1.domain.Organization;
import ru.itmo.is.lab1.realtime.ChangePublisher;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Change;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Entity;
import ru.itmo.is.lab1.repository.AddressRepository;
import ru.itmo.is.lab1.repository.CoordinatesRepository;
import ru.itmo.is.lab1.repository.OrganizationRepository;

import java.util.Collection;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/**
 * Удаление организаций вместе со связанными объектами. Координаты и адреса
 * удаляются, если на них больше не ссылается ни одна организация: общий адрес
 * другой организации при этом не пропадает.
 */
@Component
@Transactional(propagation = Propagation.MANDATORY)
public class CascadeDeleter {

    private final OrganizationRepository organizationRepository;
    private final CoordinatesRepository coordinatesRepository;
    private final AddressRepository addressRepository;
    private final ChangePublisher changePublisher;

    public CascadeDeleter(OrganizationRepository organizationRepository,
```

```
                        CoordinatesRepository coordinatesRepository,
                        AddressRepository addressRepository,
                        ChangePublisher changePublisher) {
        this.organizationRepository = organizationRepository;
        this.coordinatesRepository = coordinatesRepository;
        this.addressRepository = addressRepository;
        this.changePublisher = changePublisher;
    }

    public void deleteOrganizations(Collection<Organization> organizations) {
        if (organizations.isEmpty()) {
            return;
        }
        Set<Long> coordinatesIds = new LinkedHashSet<>();
        Set<Long> addressIds = new LinkedHashSet<>();
        List<Long> organizationIds = organizations.stream().map(Organization::getId).toList();
        for (Organization organization : organizations) {
            // getId() у ленивого прокси не инициализирует его
            coordinatesIds.add(organization.getCoordinates().getId());
            if (organization.getOfficialAddress() != null) {
                addressIds.add(organization.getOfficialAddress().getId());
            }
            if (organization.getPostalAddress() != null) {
                addressIds.add(organization.getPostalAddress().getId());
            }
        }

        organizationRepository.deleteAll(organizations);
        organizationRepository.flush();
        changePublisher.publish(Entity.ORGANIZATION, Change.DELETED, organizationIds);

        deleteUnusedRelatedObjects(coordinatesIds, addressIds);
    }

    /** Удаляет из переданных координат и адресов те, что больше никем не используются. */
    public void deleteUnusedRelatedObjects(Collection<Long> coordinatesIds, Collection<Long> addressIds) {
        List<Long> coordinatesCandidates = coordinatesIds.stream().filter(Objects::nonNull).distinct().toList();
        if (!coordinatesCandidates.isEmpty()) {
            List<Long> unused = coordinatesRepository.findUnusedIds(coordinatesCandidates);
            if (!unused.isEmpty()) {
                coordinatesRepository.deleteAllByIdInBatch(unused);
                changePublisher.publish(Entity.COORDINATES, Change.DELETED, unused);
            }
        }

        List<Long> addressCandidates = addressIds.stream().filter(Objects::nonNull).distinct().toList();
        if (!addressCandidates.isEmpty()) {
            List<Long> unused = addressRepository.findUnusedIds(addressCandidates);
            if (!unused.isEmpty()) {
                addressRepository.deleteAllByIdInBatch(unused);
                changePublisher.publish(Entity.ADDRESS, Change.DELETED, unused);
            }
        }
    }
}
```

Листинг A.29 — src/main/java/ru/itmo/is/lab1/service/DtoMapper.java

```
package ru.itmo.is.lab1.service;

import ru.itmo.is.lab1.domain.Address;
import ru.itmo.is.lab1.domain.Coordinates;
import ru.itmo.is.lab1.domain.Organization;
import ru.itmo.is.lab1.dto.AddressDto;
import ru.itmo.is.lab1.dto.CoordinatesDto;
import ru.itmo.is.lab1.dto.OrganizationDto;

/** Вызывать внутри транзакции: связи организации загружаются лениво. */
final class DtoMapper {

    private DtoMapper() {
    }

    static OrganizationDto toDto(Organization organization) {
        return new OrganizationDto(
                organization.getId(),
                organization.getName(),
                toDto(organization.getCoordinates(), null),
```

```
                organization.getCreationDate(),
                toDto(organization.getOfficialAddress(), null),
                organization.getAnnualTurnover(),
                organization.getEmployeesCount(),
                organization.getRating(),
                organization.getFullName(),
                organization.getType(),
                toDto(organization.getPostalAddress(), null));
    }

    static CoordinatesDto toDto(Coordinates coordinates, Long usages) {
        return coordinates == null ? null
                : new CoordinatesDto(coordinates.getId(), coordinates.getX(), coordinates.getY(), usages);
    }

    static AddressDto toDto(Address address, Long usages) {
        return address == null ? null : new AddressDto(address.getId(), address.getZipCode(), usages);
    }
}
```

Листинг A.30 — src/main/java/ru/itmo/is/lab1/service/OrganizationFilter.java

```
package ru.itmo.is.lab1.service;

import ru.itmo.is.lab1.domain.OrganizationType;

/** Фильтр таблицы: null — без ограничения, иначе полное совпадение. */
public record OrganizationFilter(String name,
                                 String fullName,
                                 OrganizationType type,
                                 String officialZipCode,
                                 String postalZipCode) {
}
```

Листинг A.31 — src/main/java/ru/itmo/is/lab1/service/OrganizationService.java

```
package ru.itmo.is.lab1.service;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import ru.itmo.is.lab1.domain.Address;
import ru.itmo.is.lab1.domain.Coordinates;
import ru.itmo.is.lab1.domain.Organization;
import ru.itmo.is.lab1.dto.AddressInput;
import ru.itmo.is.lab1.dto.CoordinatesInput;
import ru.itmo.is.lab1.dto.OrganizationDto;
import ru.itmo.is.lab1.dto.OrganizationRequest;
import ru.itmo.is.lab1.dto.PageDto;
import ru.itmo.is.lab1.realtime.ChangePublisher;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Change;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Entity;
import ru.itmo.is.lab1.repository.AddressRepository;
import ru.itmo.is.lab1.repository.CoordinatesRepository;
import ru.itmo.is.lab1.repository.OrganizationRepository;
import ru.itmo.is.lab1.repository.OrganizationSpecifications;
import ru.itmo.is.lab1.service.exception.InvalidInputException;
import ru.itmo.is.lab1.service.exception.NotFoundException;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import static java.util.Map.entry;

@Service
public class OrganizationService {

    public static final int MAX_PAGE_SIZE = 100;

    /** Колонка таблицы на клиенте -> свойство сущности для сортировки. */
    private static final Map<String, String> SORT_PROPERTIES = Map.ofEntries(
            entry("id", "id"),
            entry("name", "name"),
            entry("coordinates", "coordinates.x"),
```

```
        entry("creationDate", "creationDate"),
        entry("officialAddress", "officialAddress.zipCode"),
        entry("annualTurnover", "annualTurnover"),
        entry("employeesCount", "employeesCount"),
        entry("rating", "rating"),
        entry("fullName", "fullName"),
        entry("type", "type"),
        entry("postalAddress", "postalAddress.zipCode"));

private final OrganizationRepository organizationRepository;
private final CoordinatesRepository coordinatesRepository;
private final AddressRepository addressRepository;
private final CascadeDeleter cascadeDeleter;
private final ChangePublisher changePublisher;

public OrganizationService(OrganizationRepository organizationRepository,
                           CoordinatesRepository coordinatesRepository,
                           AddressRepository addressRepository,
                           CascadeDeleter cascadeDeleter,
                           ChangePublisher changePublisher) {
    this.organizationRepository = organizationRepository;
    this.coordinatesRepository = coordinatesRepository;
    this.addressRepository = addressRepository;
    this.cascadeDeleter = cascadeDeleter;
    this.changePublisher = changePublisher;
}

@Transactional(readOnly = true)
public PageDto<OrganizationDto> findPage(OrganizationFilter filter, int page, int size,
                                         String sortField, String sortDirection) {
    Map<String, String> errors = new LinkedHashMap<>();
    if (page < 0) {
        errors.put("page", "Номер страницы не может быть отрицательным");
    }
    if (size < 1 || size > MAX_PAGE_SIZE) {
        errors.put("size", "Размер страницы должен быть от 1 до " + MAX_PAGE_SIZE);
    }
    String sortProperty = SORT_PROPERTIES.get(sortField);
    if (sortProperty == null) {
        errors.put("sort", "Сортировка по полю '" + sortField + "' не поддерживается");
    }
    Sort.Direction direction = Sort.Direction.fromOptionalString(sortDirection).orElse(null);
    if (direction == null) {
        errors.put("direction", "Направление сортировки: asc или desc");
    }
    if (!errors.isEmpty()) {
        throw new InvalidInputException(errors);
    }

    // Вторичная сортировка по id делает порядок строк между страницами стабильным
    Sort sort = Sort.by(direction, sortProperty).and(Sort.by("id"));
    Page<Organization> result = organizationRepository.findAll(
            OrganizationSpecifications.matching(filter.name(), filter.fullName(), filter.type(),
                    filter.officialZipCode(), filter.postalZipCode()),
            PageRequest.of(page, size, sort));
    return new PageDto<>(result.map(DtoMapper::toDto).getContent(), result.getNumber(), result.getSize(),
            result.getTotalElements(), result.getTotalPages());
}

@Transactional(readOnly = true)
public OrganizationDto get(long id) {
    return DtoMapper.toDto(organizationRepository.findWithRelationsById(id)
            .orElseThrow(() -> NotFoundException.organization(id)));
}

@Transactional
public OrganizationDto create(OrganizationRequest request) {
    Relations relations = resolveRelations(request);
    Organization organization = new Organization();
    applyRequest(organization, request, relations);
    organizationRepository.save(organization);
    changePublisher.publish(Entity.ORGANIZATION, Change.CREATED, organization.getId());
    return DtoMapper.toDto(organization);
}

@Transactional
public OrganizationDto update(long id, OrganizationRequest request) {
    Organization organization = organizationRepository.findWithRelationsById(id)
```

```
                .orElseThrow(() -> NotFoundException.organization(id));
        Relations relations = resolveRelations(request);

        Long oldCoordinatesId = organization.getCoordinates().getId();
        List<Long> oldAddressIds = new ArrayList<>();
        if (organization.getOfficialAddress() != null) {
            oldAddressIds.add(organization.getOfficialAddress().getId());
        }
        if (organization.getPostalAddress() != null) {
            oldAddressIds.add(organization.getPostalAddress().getId());
        }

        applyRequest(organization, request, relations);
        organizationRepository.flush();
        changePublisher.publish(Entity.ORGANIZATION, Change.UPDATED, id);

        // Прежние координаты/адреса, на которые больше никто не ссылается, удаляются
        cascadeDeleter.deleteUnusedRelatedObjects(List.of(oldCoordinatesId), oldAddressIds);
        return DtoMapper.toDto(organization);
    }

    @Transactional
    public void delete(long id) {
        Organization organization = organizationRepository.findById(id)
                .orElseThrow(() -> NotFoundException.organization(id));
        cascadeDeleter.deleteOrganizations(List.of(organization));
    }

    private void applyRequest(Organization organization, OrganizationRequest request, Relations relations) {
        organization.setName(request.name());
        organization.setFullName(request.fullName());
        organization.setType(request.type());
        organization.setEmployeesCount(request.employeesCount());
        organization.setAnnualTurnover(request.annualTurnover());
        organization.setRating(request.rating());
        organization.setCoordinates(saveIfNew(relations.coordinates()));
        organization.setOfficialAddress(saveIfNew(relations.officialAddress()));
        organization.setPostalAddress(saveIfNew(relations.postalAddress()));
    }

    /**
     * Проверяет ссылки на вспомогательные объекты. Новые объекты возвращаются
     * несохранёнными: в БД они попадут, только если все поля корректны.
     */
    private Relations resolveRelations(OrganizationRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();
        if (request.rating() != null && !Float.isFinite(request.rating())) {
            errors.put("rating", "Рейтинг должен быть конечным числом");
        }
        Coordinates coordinates = resolveCoordinates(request.coordinates(), errors);
        Address officialAddress = resolveAddress(request.officialAddress(), "officialAddress", errors);
        Address postalAddress = resolveAddress(request.postalAddress(), "postalAddress", errors);
        if (!errors.isEmpty()) {
            throw new InvalidInputException(errors);
        }
        return new Relations(coordinates, officialAddress, postalAddress);
    }

    private Coordinates resolveCoordinates(CoordinatesInput input, Map<String, String> errors) {
        if (input == null) {
            errors.put("coordinates", "Координаты обязательны");
            return null;
        }
        if (input.id() != null) {
            Coordinates existing = coordinatesRepository.findById(input.id()).orElse(null);
            if (existing == null) {
                errors.put("coordinates", "Координаты с id = " + input.id() + " не найдены (возможно,
удалены)");
            }
            return existing;
        }

        int errorsBefore = errors.size();
        if (input.x() == null) {
            errors.put("coordinates.x", "Укажите X");
        } else if (!Double.isFinite(input.x())) {
            errors.put("coordinates.x", "X должен быть конечным числом");
        } else if (input.x() > Coordinates.MAX_X) {
```

```
            errors.put("coordinates.x", "X не может быть больше 549");
        }
        if (input.y() == null) {
            errors.put("coordinates.y", "Укажите Y");
        } else if (!Double.isFinite(input.y())) {
            errors.put("coordinates.y", "Y должен быть конечным числом");
        }
        return errors.size() == errorsBefore ? new Coordinates(input.x(), input.y()) : null;
    }

    private Address resolveAddress(AddressInput input, String field, Map<String, String> errors) {
        if (input == null) {
            return null;
        }
        if (input.id() != null) {
            Address existing = addressRepository.findById(input.id()).orElse(null);
            if (existing == null) {
                errors.put(field, "Адрес с id = " + input.id() + " не найден (возможно, удалён)");
            }
            return existing;
        }
        if (input.zipCode() == null) {
            errors.put(field + ".zipCode", "Укажите почтовый индекс");
            return null;
        }
        return new Address(input.zipCode());
    }

    private Coordinates saveIfNew(Coordinates coordinates) {
        if (coordinates.getId() != null) {
            return coordinates;
        }
        Coordinates saved = coordinatesRepository.save(coordinates);
        changePublisher.publish(Entity.COORDINATES, Change.CREATED, saved.getId());
        return saved;
    }

    private Address saveIfNew(Address address) {
        if (address == null || address.getId() != null) {
            return address;
        }
        Address saved = addressRepository.save(address);
        changePublisher.publish(Entity.ADDRESS, Change.CREATED, saved.getId());
        return saved;
    }

    private record Relations(Coordinates coordinates, Address officialAddress, Address postalAddress) {
    }
}
```

Листинг A.32 — src/main/java/ru/itmo/is/lab1/service/RelatedObjectService.java

```
package ru.itmo.is.lab1.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import ru.itmo.is.lab1.dto.AddressDto;
import ru.itmo.is.lab1.dto.CoordinatesDto;
import ru.itmo.is.lab1.repository.AddressRepository;
import ru.itmo.is.lab1.repository.CoordinatesRepository;
import ru.itmo.is.lab1.repository.OrganizationRepository;
import ru.itmo.is.lab1.service.exception.NotFoundException;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Вспомогательные объекты (координаты и адреса), к которым можно привязать организацию. */
@Service
public class RelatedObjectService {

    private final CoordinatesRepository coordinatesRepository;
    private final AddressRepository addressRepository;
    private final OrganizationRepository organizationRepository;
    private final CascadeDeleter cascadeDeleter;

    public RelatedObjectService(CoordinatesRepository coordinatesRepository,
                                AddressRepository addressRepository,
```

```
                                OrganizationRepository organizationRepository,
                                CascadeDeleter cascadeDeleter) {
        this.coordinatesRepository = coordinatesRepository;
        this.addressRepository = addressRepository;
        this.organizationRepository = organizationRepository;
        this.cascadeDeleter = cascadeDeleter;
    }

    @Transactional(readOnly = true)
    public List<CoordinatesDto> listCoordinates() {
        Map<Long, Long> usages = toUsageMap(organizationRepository.countUsagesByCoordinates());
        return coordinatesRepository.findAllByOrderById().stream()
                .map(c -> DtoMapper.toDto(c, usages.getOrDefault(c.getId(), 0L)))
                .toList();
    }

    @Transactional(readOnly = true)
    public List<AddressDto> listAddresses() {
        Map<Long, Long> usages = toUsageMap(organizationRepository.countUsagesByAddress());
        return addressRepository.findAllByOrderById().stream()
                .map(a -> DtoMapper.toDto(a, usages.getOrDefault(a.getId(), 0L)))
                .toList();
    }

    /** Удаляет координаты вместе со всеми организациями, которые на них ссылаются. */
    @Transactional
    public void deleteCoordinates(long id) {
        if (!coordinatesRepository.existsById(id)) {
            throw new NotFoundException("Координаты с id = " + id + " не найдены");
        }
        cascadeDeleter.deleteOrganizations(organizationRepository.findAllByCoordinatesId(id));
        cascadeDeleter.deleteUnusedRelatedObjects(List.of(id), List.of());
    }

    /** Удаляет адрес вместе со всеми организациями, которые на него ссылаются. */
    @Transactional
    public void deleteAddress(long id) {
        if (!addressRepository.existsById(id)) {
            throw new NotFoundException("Адрес с id = " + id + " не найден");
        }
        cascadeDeleter.deleteOrganizations(organizationRepository.findAllByAddressId(id));
        cascadeDeleter.deleteUnusedRelatedObjects(List.of(), List.of(id));
    }

    private static Map<Long, Long> toUsageMap(List<Object[]> rows) {
        Map<Long, Long> usages = new HashMap<>();
        for (Object[] row : rows) {
            usages.put(((Number) row[0]).longValue(), ((Number) row[1]).longValue());
        }
        return usages;
    }
}
```

Листинг A.33 — src/main/java/ru/itmo/is/lab1/service/SpecialOperationsService.java

```
package ru.itmo.is.lab1.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import ru.itmo.is.lab1.domain.OrganizationType;
import ru.itmo.is.lab1.dto.AbsorbResult;
import ru.itmo.is.lab1.dto.OrganizationDto;
import ru.itmo.is.lab1.realtime.ChangePublisher;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Change;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Entity;
import ru.itmo.is.lab1.repository.OrganizationRepository;
import ru.itmo.is.lab1.repository.OrganizationRepository.RelationIds;
import ru.itmo.is.lab1.repository.SpecialOperationsRepository;
import ru.itmo.is.lab1.service.exception.InvalidInputException;
import ru.itmo.is.lab1.service.exception.NotFoundException;

import java.util.ArrayList;
import java.util.List;

/** Специальные операции. Сама логика реализована функциями БД. */
@Service
public class SpecialOperationsService {
```

```
private final SpecialOperationsRepository specialOperationsRepository;
private final OrganizationRepository organizationRepository;
private final CascadeDeleter cascadeDeleter;
private final ChangePublisher changePublisher;

public SpecialOperationsService(SpecialOperationsRepository specialOperationsRepository,
                                OrganizationRepository organizationRepository,
                                CascadeDeleter cascadeDeleter,
                                ChangePublisher changePublisher) {
    this.specialOperationsRepository = specialOperationsRepository;
    this.organizationRepository = organizationRepository;
    this.cascadeDeleter = cascadeDeleter;
    this.changePublisher = changePublisher;
}

@Transactional(readOnly = true)
public long countByType(OrganizationType type) {
    requireType(type);
    return specialOperationsRepository.countByType(type);
}

@Transactional(readOnly = true)
public long countFullNameGreaterThan(String fullName) {
    if (fullName == null) {
        throw new InvalidInputException("fullName", "Укажите значение fullName для сравнения");
    }
    return specialOperationsRepository.countFullNameGreaterThan(fullName);
}

@Transactional(readOnly = true)
public List<OrganizationDto> findTypeLessThan(OrganizationType type) {
    requireType(type);
    return specialOperationsRepository.findTypeLessThan(type).stream()
            .map(DtoMapper::toDto)
            .toList();
}

@Transactional
public OrganizationDto addEmployee(long organizationId) {
    if (!organizationRepository.existsById(organizationId)) {
        throw NotFoundException.organization(organizationId);
    }
    specialOperationsRepository.addEmployee(organizationId);
    changePublisher.publish(Entity.ORGANIZATION, Change.UPDATED, organizationId);
    return loadFresh(organizationId);
}

@Transactional
public AbsorbResult absorb(long absorberId, long absorbedId) {
    if (absorberId == absorbedId) {
        throw new InvalidInputException("absorbedId", "Организация не может поглотить саму себя");
    }
    if (!organizationRepository.existsById(absorberId)) {
        throw new InvalidInputException("absorberId",
                "Поглощающая организация с id = " + absorberId + " не найдена");
    }
    RelationIds absorbedRelations = organizationRepository.findRelationIds(absorbedId)
            .orElseThrow(() -> new InvalidInputException("absorbedId",
                    "Поглощаемая организация с id = " + absorbedId + " не найдена"));
    long employeesBefore = organizationRepository.findEmployeesCount(absorberId).orElseThrow();

    long employeesAfter = specialOperationsRepository.absorb(absorberId, absorbedId);

    changePublisher.publish(Entity.ORGANIZATION, Change.DELETED, absorbedId);
    changePublisher.publish(Entity.ORGANIZATION, Change.UPDATED, absorberId);
    List<Long> addressIds = new ArrayList<>();
    addressIds.add(absorbedRelations.getOfficialAddressId());
    addressIds.add(absorbedRelations.getPostalAddressId());
    cascadeDeleter.deleteUnusedRelatedObjects(List.of(absorbedRelations.getCoordinatesId()), addressIds);

    return new AbsorbResult(loadFresh(absorberId), absorbedId, employeesAfter - employeesBefore);
}

private OrganizationDto loadFresh(long id) {
    return DtoMapper.toDto(organizationRepository.findWithRelationsById(id)
            .orElseThrow(() -> NotFoundException.organization(id)));
}
```

```
    private static void requireType(OrganizationType type) {
        if (type == null) {
            throw new InvalidInputException("type", "Выберите тип организации");
        }
    }
}
```

Листинг A.34 — src/main/java/ru/itmo/is/lab1/service/exception/InvalidInputException.java

```
package ru.itmo.is.lab1.service.exception;

import java.util.Map;

/** Ошибка пользовательского ввода, привязанная к конкретным полям формы. */
public class InvalidInputException extends RuntimeException {

    private final Map<String, String> fieldErrors;

    public InvalidInputException(Map<String, String> fieldErrors) {
        super("Проверьте правильность заполнения полей");
        this.fieldErrors = Map.copyOf(fieldErrors);
    }

    public InvalidInputException(String field, String message) {
        super(message);
        this.fieldErrors = Map.of(field, message);
    }

    public Map<String, String> getFieldErrors() {
        return fieldErrors;
    }
}
```

Листинг A.35 — src/main/java/ru/itmo/is/lab1/service/exception/NotFoundException.java

```
package ru.itmo.is.lab1.service.exception;

public class NotFoundException extends RuntimeException {

    public NotFoundException(String message) {
        super(message);
    }

    public static NotFoundException organization(long id) {
        return new NotFoundException("Организация с id = " + id + " не найдена (возможно, её удалил другой
пользователь)");
    }
}
```

Листинг A.36 — src/main/java/ru/itmo/is/lab1/realtime/ChangeBroadcaster.java

```
package ru.itmo.is.lab1.realtime;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

/**
 * Рассылает изменения только после успешного коммита, чтобы клиенты,
 * перезапросив данные, гарантированно увидели новое состояние БД.
 */
@Component
public class ChangeBroadcaster {

    private final UpdatesWebSocketHandler webSocketHandler;
    private final ObjectMapper objectMapper;

    public ChangeBroadcaster(UpdatesWebSocketHandler webSocketHandler, ObjectMapper objectMapper) {
        this.webSocketHandler = webSocketHandler;
        this.objectMapper = objectMapper;
    }
```

```
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution = true)
    public void onChange(EntityChangedEvent event) throws JsonProcessingException {
        webSocketHandler.broadcast(objectMapper.writeValueAsString(event));
    }
}
```

Листинг A.37 — src/main/java/ru/itmo/is/lab1/realtime/ChangePublisher.java

```
package ru.itmo.is.lab1.realtime;

import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Component;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Change;
import ru.itmo.is.lab1.realtime.EntityChangedEvent.Entity;

import java.util.Collection;
import java.util.List;

@Component
public class ChangePublisher {

    private final ApplicationEventPublisher eventPublisher;

    public ChangePublisher(ApplicationEventPublisher eventPublisher) {
        this.eventPublisher = eventPublisher;
    }

    public void publish(Entity entity, Change change, Collection<Long> ids) {
        if (!ids.isEmpty()) {
            eventPublisher.publishEvent(new EntityChangedEvent(entity, change, List.copyOf(ids)));
        }
    }

    public void publish(Entity entity, Change change, long id) {
        publish(entity, change, List.of(id));
    }
}
```

Листинг A.38 — src/main/java/ru/itmo/is/lab1/realtime/EntityChangedEvent.java

```
package ru.itmo.is.lab1.realtime;

import java.util.List;

/** Сообщение, которое рассылается всем подключённым клиентам после фиксации транзакции. */
public record EntityChangedEvent(Entity entity, Change change, List<Long> ids) {

    public enum Entity {
        ORGANIZATION,
        COORDINATES,
        ADDRESS
    }

    public enum Change {
        CREATED,
        UPDATED,
        DELETED
    }
}
```

Листинг A.39 — src/main/java/ru/itmo/is/lab1/realtime/UpdatesWebSocketHandler.java

```
package ru.itmo.is.lab1.realtime;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.CloseStatus;
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.handler.ConcurrentWebSocketSessionDecorator;
import org.springframework.web.socket.handler.TextWebSocketHandler;

import java.io.IOException;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
```

```
/** Держит открытые соединения браузеров и рассылает им уведомления об изменениях. */
@Component
public class UpdatesWebSocketHandler extends TextWebSocketHandler {

    private static final Logger log = LoggerFactory.getLogger(UpdatesWebSocketHandler.class);
    private static final int SEND_TIME_LIMIT_MS = 5_000;
    private static final int BUFFER_SIZE_LIMIT = 512 * 1024;

    private final Map<String, WebSocketSession> sessions = new ConcurrentHashMap<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) {
        // Декоратор сериализует отправку: WebSocketSession не потокобезопасен
        sessions.put(session.getId(),
                new ConcurrentWebSocketSessionDecorator(session, SEND_TIME_LIMIT_MS, BUFFER_SIZE_LIMIT));
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
        sessions.remove(session.getId());
    }

    @Override
    public void handleTransportError(WebSocketSession session, Throwable exception) {
        sessions.remove(session.getId());
    }

    public void broadcast(String payload) {
        TextMessage message = new TextMessage(payload);
        sessions.values().forEach(session -> {
            try {
                if (session.isOpen()) {
                    session.sendMessage(message);
                }
            } catch (IOException | IllegalStateException e) {
                log.debug("Не удалось отправить уведомление в сессию {}: {}", session.getId(), e.getMessage());
                sessions.remove(session.getId());
            }
        });
    }
}
```

Листинг A.40 — src/main/java/ru/itmo/is/lab1/realtime/WebSocketConfig.java

```
package ru.itmo.is.lab1.realtime;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final UpdatesWebSocketHandler updatesHandler;

    public WebSocketConfig(UpdatesWebSocketHandler updatesHandler) {
        this.updatesHandler = updatesHandler;
    }

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(updatesHandler, "/ws/updates");
    }
}
```

Листинг A.41 — src/main/java/ru/itmo/is/lab1/web/ApiExceptionHandler.java

```
package ru.itmo.is.lab1.web;

import com.fasterxml.jackson.databind.JsonMappingException;
import com.fasterxml.jackson.databind.exc.InvalidFormatException;
import com.fasterxml.jackson.databind.exc.MismatchedInputException;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import org.postgresql.util.PSQLException;
```

```
import org.postgresql.util.ServerErrorMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataAccessException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.transaction.TransactionSystemException;
import org.springframework.validation.BindException;
import org.springframework.validation.FieldError;
import org.springframework.web.HttpRequestMethodNotSupportedException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;
import org.springframework.web.servlet.resource.NoResourceFoundException;
import ru.itmo.is.lab1.domain.OrganizationType;
import ru.itmo.is.lab1.dto.ApiError;
import ru.itmo.is.lab1.service.exception.InvalidInputException;
import ru.itmo.is.lab1.service.exception.NotFoundException;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.stream.Collectors;

import static java.util.Map.entry;

@RestControllerAdvice
public class ApiExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);
    private static final String CHECK_FIELDS = "Проверьте правильность заполнения полей";
    private static final String NUMERIC_OVERFLOW =
            "Слишком большое число: результат операции не помещается в допустимый диапазон (bigint)";

    /** Ограничения БД (V1__schema.sql) -> понятное пользователю сообщение. */
    private static final Map<String, Map.Entry<String, String>> DB_CONSTRAINTS = Map.ofEntries(
            entry("coordinates_x_max", entry("coordinates.x", "X не может быть больше 549")),
            entry("coordinates_x_finite", entry("coordinates.x", "X должен быть конечным числом")),
            entry("coordinates_y_finite", entry("coordinates.y", "Y должен быть конечным числом")),
            entry("organization_name_not_empty", entry("name", "Название не может быть пустым")),
            entry("organization_annual_turnover_positive", entry("annualTurnover", "Годовой оборот должен быть
больше 0")),
            entry("organization_employees_count_positive", entry("employeesCount", "Количество сотрудников
должно быть больше 0")),
            entry("organization_rating_positive", entry("rating", "Рейтинг должен быть больше 0")),
            entry("organization_type_valid", entry("type", "Недопустимый тип организации")),
            entry("organization_coordinates_fk", entry("coordinates", "Выбранные координаты уже удалены")),
            entry("organization_official_address_fk", entry("officialAddress", "Выбранный адрес уже удалён")),
            entry("organization_postal_address_fk", entry("postalAddress", "Выбранный адрес уже удалён")));

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ApiError> notFound(NotFoundException e) {
        return respond(HttpStatus.NOT_FOUND, e.getMessage(), Map.of());
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ApiError> invalidInput(InvalidInputException e) {
        return respond(HttpStatus.BAD_REQUEST, e.getMessage(), e.getFieldErrors());
    }

    /** Ошибки Bean Validation для тела запроса (@Valid) и привязки параметров. */
    @ExceptionHandler(BindException.class)
    public ResponseEntity<ApiError> bindingFailed(BindException e) {
        Map<String, String> errors = new LinkedHashMap<>();
        for (FieldError error : e.getFieldErrors()) {
            errors.putIfAbsent(error.getField(), error.isBindingFailure()
                    ? "Некорректное значение" : error.getDefaultMessage());
        }
        e.getGlobalErrors().forEach(error -> errors.putIfAbsent(error.getObjectName(),
error.getDefaultMessage()));
        return respond(HttpStatus.BAD_REQUEST, CHECK_FIELDS, errors);
    }

    /** Ошибки Bean Validation на уровне ORM (при сохранении сущности). */
    @ExceptionHandler({ConstraintViolationException.class, TransactionSystemException.class})
```

```
    public ResponseEntity<ApiError> entityValidationFailed(Exception e) {
        ConstraintViolationException violation = findCause(e, ConstraintViolationException.class);
        if (violation == null) {
            return unexpected(e);
        }
        Map<String, String> errors = new LinkedHashMap<>();
        for (ConstraintViolation<?> v : violation.getConstraintViolations()) {
            errors.putIfAbsent(v.getPropertyPath().toString(), v.getMessage());
        }
        return respond(HttpStatus.BAD_REQUEST, CHECK_FIELDS, errors);
    }

    /** Некорректный JSON: строка вместо числа, неизвестный тип, дробное число вместо целого и т.п. */
    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ApiError> unreadableBody(HttpMessageNotReadableException e) {
        JsonMappingException mapping = findCause(e, JsonMappingException.class);
        if (mapping == null || mapping.getPath().isEmpty()) {
            return respond(HttpStatus.BAD_REQUEST, "Тело запроса не является корректным JSON", Map.of());
        }
        String field = mapping.getPath().stream()
                .map(ref -> ref.getFieldName() != null ? ref.getFieldName() : String.valueOf(ref.getIndex()))
                .collect(Collectors.joining("."));
        Class<?> targetType = mapping instanceof MismatchedInputException mismatch ? mismatch.getTargetType() :
null;
        return respond(HttpStatus.BAD_REQUEST, CHECK_FIELDS, Map.of(field, describeExpectedType(targetType,
mapping)));
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiError> typeMismatch(MethodArgumentTypeMismatchException e) {
        String message = "Параметр '" + e.getName() + "': " + describeExpectedType(e.getRequiredType(), null);
        return respond(HttpStatus.BAD_REQUEST, message, Map.of(e.getName(), message));
    }

    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ApiError> missingParameter(MissingServletRequestParameterException e) {
        String message = "Не указан параметр '" + e.getParameterName() + "'";
        return respond(HttpStatus.BAD_REQUEST, message, Map.of(e.getParameterName(), message));
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ApiError> integrityViolation(DataIntegrityViolationException e) {
        String constraint = findConstraintName(e);
        Map.Entry<String, String> known = constraint == null ? null : DB_CONSTRAINTS.get(constraint);
        if (known != null) {
            return respond(HttpStatus.BAD_REQUEST, known.getValue(), Map.of(known.getKey(), known.getValue()));
        }
        // Spring относит ошибки данных SQLSTATE 22xxx к нарушению целостности; для пользователя это ошибка
ввода
        PSQLException psql = findCause(e, PSQLException.class);
        if (psql != null && "22003".equals(psql.getSQLState())) {
            return respond(HttpStatus.BAD_REQUEST,
                    NUMERIC_OVERFLOW, Map.of());
        }
        if (psql != null && psql.getSQLState() != null && psql.getSQLState().startsWith("22")) {
            return respond(HttpStatus.BAD_REQUEST, databaseMessage(e), Map.of());
        }
        log.warn("Нарушено ограничение БД", e);
        return respond(HttpStatus.CONFLICT, "Операция нарушает ограничения базы данных: " + databaseMessage(e),
Map.of());
    }

    /** Ошибки, выброшенные функциями БД (RAISE EXCEPTION), и прочие ошибки доступа к данным. */
    @ExceptionHandler(DataAccessException.class)
    public ResponseEntity<ApiError> dataAccess(DataAccessException e) {
        PSQLException psql = findCause(e, PSQLException.class);
        if (psql != null && psql.getServerErrorMessage() != null) {
            String sqlState = psql.getSQLState();
            // P0002 - no_data_found, 22xxx - некорректные данные, P0001 - raise_exception
            if ("P0002".equals(sqlState)) {
                return respond(HttpStatus.NOT_FOUND, databaseMessage(e), Map.of());
            }
            if ("22003".equals(sqlState)) {
                return respond(HttpStatus.BAD_REQUEST,
                        NUMERIC_OVERFLOW, Map.of());
            }
            if (sqlState != null && (sqlState.startsWith("22") || sqlState.startsWith("P0"))) {
                return respond(HttpStatus.BAD_REQUEST, databaseMessage(e), Map.of());
```

```
            }
        }
        return unexpected(e);
    }

    @ExceptionHandler(HttpRequestMethodNotSupportedException.class)
    public ResponseEntity<ApiError> methodNotSupported(HttpRequestMethodNotSupportedException e) {
        return respond(HttpStatus.METHOD_NOT_ALLOWED, "Метод " + e.getMethod() + " не поддерживается",
Map.of());
    }

    @ExceptionHandler(NoResourceFoundException.class)
    public ResponseEntity<ApiError> noResource(NoResourceFoundException e) {
        return respond(HttpStatus.NOT_FOUND, "Ресурс не найден: /" + e.getResourcePath(), Map.of());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> unexpected(Exception e) {
        log.error("Необработанная ошибка", e);
        return respond(HttpStatus.INTERNAL_SERVER_ERROR, "Внутренняя ошибка сервера", Map.of());
    }

    private static String describeExpectedType(Class<?> type, JsonMappingException cause) {
        if (type == null) {
            return cause instanceof InvalidFormatException ? "Некорректное значение" : "Некорректный формат
значения";
        }
        if (type.isEnum()) {
            String allowed =
Arrays.stream(type.getEnumConstants()).map(String::valueOf).collect(Collectors.joining(", "));
            return (type == OrganizationType.class ? "Недопустимый тип организации" : "Недопустимое значение")
                    + ". Допустимые значения: " + allowed;
        }
        if (type == Long.class || type == long.class || type == Integer.class || type == int.class) {
            return "Ожидается целое число в допустимом диапазоне";
        }
        if (type == Double.class || type == double.class || type == Float.class || type == float.class) {
            return "Ожидается число";
        }
        if (type == String.class) {
            return "Ожидается строка";
        }
        return "Некорректное значение";
    }

    private static String findConstraintName(Throwable e) {
        org.hibernate.exception.ConstraintViolationException hibernate =
                findCause(e, org.hibernate.exception.ConstraintViolationException.class);
        if (hibernate != null && hibernate.getConstraintName() != null) {
            return hibernate.getConstraintName();
        }
        PSQLException psql = findCause(e, PSQLException.class);
        ServerErrorMessage server = psql == null ? null : psql.getServerErrorMessage();
        return server == null ? null : server.getConstraint();
    }

    private static String databaseMessage(Throwable e) {
        PSQLException psql = findCause(e, PSQLException.class);
        if (psql != null && psql.getServerErrorMessage() != null) {
            return psql.getServerErrorMessage().getMessage();
        }
        return e.getMessage();
    }

    private static <T extends Throwable> T findCause(Throwable e, Class<T> type) {
        for (Throwable t = e; t != null; t = t.getCause() == t ? null : t.getCause()) {
            if (type.isInstance(t)) {
                return type.cast(t);
            }
        }
        return null;
    }

    private static ResponseEntity<ApiError> respond(HttpStatus status, String message, Map<String, String>
fieldErrors) {
        return ResponseEntity.status(status).body(new ApiError(status.value(), message, fieldErrors));
    }
}
```

Листинг А.42 — src/main/java/ru/itmo/is/lab1/web/OrganizationController.java

```
package ru.itmo.is.lab1.web;

import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
import ru.itmo.is.lab1.domain.OrganizationType;
import ru.itmo.is.lab1.dto.OrganizationDto;
import ru.itmo.is.lab1.dto.OrganizationRequest;
import ru.itmo.is.lab1.dto.PageDto;
import ru.itmo.is.lab1.service.OrganizationFilter;
import ru.itmo.is.lab1.service.OrganizationService;

@RestController
@RequestMapping("/api/organizations")
public class OrganizationController {

    private final OrganizationService organizationService;

    public OrganizationController(OrganizationService organizationService) {
        this.organizationService = organizationService;
    }

    @GetMapping
    public PageDto<OrganizationDto> list(@RequestParam(defaultValue = "0") int page,
                                         @RequestParam(defaultValue = "10") int size,
                                         @RequestParam(defaultValue = "id") String sort,
                                         @RequestParam(defaultValue = "asc") String direction,
                                         @RequestParam(required = false) String name,
                                         @RequestParam(required = false) String fullName,
                                         @RequestParam(required = false) OrganizationType type,
                                         @RequestParam(required = false) String officialZipCode,
                                         @RequestParam(required = false) String postalZipCode) {
        OrganizationFilter filter = new OrganizationFilter(name, fullName, type, officialZipCode,
postalZipCode);
        return organizationService.findPage(filter, page, size, sort, direction);
    }

    @GetMapping("/{id}")
    public OrganizationDto get(@PathVariable long id) {
        return organizationService.get(id);
    }

    @PostMapping
    public ResponseEntity<OrganizationDto> create(@Valid @RequestBody OrganizationRequest request) {
        OrganizationDto created = organizationService.create(request);
        return ResponseEntity
                .created(ServletUriComponentsBuilder.fromCurrentRequest().path("/{id}").build(created.id()))
                .body(created);
    }

    @PutMapping("/{id}")
    public OrganizationDto update(@PathVariable long id, @Valid @RequestBody OrganizationRequest request) {
        return organizationService.update(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable long id) {
        organizationService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Листинг А.43 — src/main/java/ru/itmo/is/lab1/web/RelatedObjectController.java

```
package ru.itmo.is.lab1.web;

import org.springframework.http.ResponseEntity;
```

```
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import ru.itmo.is.lab1.dto.AddressDto;
import ru.itmo.is.lab1.dto.CoordinatesDto;
import ru.itmo.is.lab1.service.RelatedObjectService;

import java.util.List;

@RestController
@RequestMapping("/api")
public class RelatedObjectController {

    private final RelatedObjectService relatedObjectService;

    public RelatedObjectController(RelatedObjectService relatedObjectService) {
        this.relatedObjectService = relatedObjectService;
    }

    @GetMapping("/coordinates")
    public List<CoordinatesDto> coordinates() {
        return relatedObjectService.listCoordinates();
    }

    @DeleteMapping("/coordinates/{id}")
    public ResponseEntity<Void> deleteCoordinates(@PathVariable long id) {
        relatedObjectService.deleteCoordinates(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/addresses")
    public List<AddressDto> addresses() {
        return relatedObjectService.listAddresses();
    }

    @DeleteMapping("/addresses/{id}")
    public ResponseEntity<Void> deleteAddress(@PathVariable long id) {
        relatedObjectService.deleteAddress(id);
        return ResponseEntity.noContent().build();
    }
}
```

Листинг A.44 — src/main/java/ru/itmo/is/lab1/web/SpecialOperationsController.java

```
package ru.itmo.is.lab1.web;

import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import ru.itmo.is.lab1.domain.OrganizationType;
import ru.itmo.is.lab1.dto.AbsorbRequest;
import ru.itmo.is.lab1.dto.AbsorbResult;
import ru.itmo.is.lab1.dto.CountResult;
import ru.itmo.is.lab1.dto.OrganizationDto;
import ru.itmo.is.lab1.service.SpecialOperationsService;

import java.util.List;

@RestController
@RequestMapping("/api/special")
public class SpecialOperationsController {

    private final SpecialOperationsService specialOperationsService;

    public SpecialOperationsController(SpecialOperationsService specialOperationsService) {
        this.specialOperationsService = specialOperationsService;
    }

    @GetMapping("/count-by-type")
    public CountResult countByType(@RequestParam OrganizationType type) {
        return new CountResult(specialOperationsService.countByType(type));
```

```
    }

    @GetMapping("/count-full-name-greater")
    public CountResult countFullNameGreater(@RequestParam String fullName) {
        return new CountResult(specialOperationsService.countFullNameGreaterThan(fullName));
    }

    @GetMapping("/type-less-than")
    public List<OrganizationDto> typeLessThan(@RequestParam OrganizationType type) {
        return specialOperationsService.findTypeLessThan(type);
    }

    @PostMapping("/organizations/{id}/employees")
    public OrganizationDto addEmployee(@PathVariable long id) {
        return specialOperationsService.addEmployee(id);
    }

    @PostMapping("/absorb")
    public AbsorbResult absorb(@Valid @RequestBody AbsorbRequest request) {
        return specialOperationsService.absorb(request.absorberId(), request.absorbedId());
    }
}
```

Листинг A.45 — src/main/resources/static/create.html

```
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Создание организации — ИС «Организации»</title>
    <link rel="stylesheet" href="css/app.css">
    <script type="module" src="js/pages/create.js"></script>
</head>
<body>
<main class="page page--narrow">
    <div class="page-head">
        <div>
            <h1>Новая организация</h1>
            <div class="muted">
                Координаты и адреса можно создать заново или выбрать из уже существующих в системе.
                Поля со звёздочкой обязательны.
            </div>
        </div>
    </div>
    <div class="card" id="app"></div>
</main>
</body>
</html>
```

Листинг A.46 — src/main/resources/static/index.html

```
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Организации — ИС «Организации»</title>
    <link rel="stylesheet" href="css/app.css">
    <script type="module" src="js/pages/list.js"></script>
</head>
<body>
<main class="page">
    <div class="page-head">
        <div>
            <h1>Организации</h1>
            <div class="muted">Таблица обновляется автоматически, когда данные меняют другие пользователи</div>
        </div>
        <a class="btn btn--primary" href="create.html">+ Создать организацию</a>
    </div>
    <div id="app"></div>
</main>
</body>
</html>
```

Листинг А.47 — src/main/resources/static/related.html

```
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Координаты и адреса — ИС «Организации»</title>
    <link rel="stylesheet" href="css/app.css">
    <script type="module" src="js/pages/related.js"></script>
</head>
<body>
<main class="page">
    <div class="page-head">
        <div>
            <h1>Координаты и адреса</h1>
            <div class="muted">
                Вспомогательные объекты создаются вместе с организациями, и их можно выбрать при создании новой
                организации. Удаление такого объекта удаляет все организации, которые на него ссылаются.
            </div>
        </div>
    </div>
    <div class="grid-2" id="app"></div>
</main>
</body>
</html>
```

Листинг А.48 — src/main/resources/static/special.html

```
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Специальные операции — ИС «Организации»</title>
    <link rel="stylesheet" href="css/app.css">
    <script type="module" src="js/pages/special.js"></script>
</head>
<body>
<main class="page">
    <div class="page-head">
        <div>
            <h1>Специальные операции</h1>
            <div class="muted">Операции выполняются функциями базы данных на сервере</div>
        </div>
    </div>
    <div class="grid-2 grid-2--fixed" id="app"></div>
</main>
</body>
</html>
```

Листинг А.49 — src/main/resources/static/view.html

```
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Организация по ID — ИС «Организации»</title>
    <link rel="stylesheet" href="css/app.css">
    <script type="module" src="js/pages/view.js"></script>
</head>
<body>
<main class="page page--narrow">
    <div class="page-head">
        <div>
            <h1>Информация об организации</h1>
            <div class="muted">Введите ID, чтобы увидеть организацию и связанные с ней объекты</div>
        </div>
    </div>
    <div id="app"></div>
</main>
</body>
</html>
```

Листинг А.50 — src/main/resources/static/js/api.js

```
// Клиент REST API. Все операции выполняются на сервере.

export class ApiError extends Error {
    constructor(status, message, fieldErrors = {}) {
        super(message);
        this.status = status;
        this.fieldErrors = fieldErrors;
    }
}

async function request(method, url, body) {
    const options = {method, headers: {Accept: 'application/json'}};
    if (body !== undefined) {
        options.headers['Content-Type'] = 'application/json';
        options.body = JSON.stringify(body);
    }

    let response;
    try {
        response = await fetch(url, options);
    } catch {
        throw new ApiError(0, 'Сервер недоступен. Проверьте подключение и повторите попытку.');
    }

    const text = await response.text();
    let data = null;
    if (text) {
        try {
            data = JSON.parse(text);
        } catch {
            data = null;
        }
    }
    if (!response.ok) {
        throw new ApiError(response.status, data?.message || `Ошибка сервера (${response.status})`,
            data?.fieldErrors || {});
    }
    return data;
}

/** @param keepEmpty передавать пустые строки (фильтр «значение равно пустой строке») */
function query(params, keepEmpty = false) {
    const search = new URLSearchParams();
    for (const [key, value] of Object.entries(params)) {
        if (value !== undefined && value !== null && (keepEmpty || value !== '')) {
            search.set(key, value);
        }
    }
    const s = search.toString();
    return s ? `?${s}` : '';
}

export const api = {
    listOrganizations: (params) => request('GET', `api/organizations${query(params, true)}`),
    getOrganization: (id) => request('GET', `api/organizations/${encodeURIComponent(id)}`),
    createOrganization: (body) => request('POST', 'api/organizations', body),
    updateOrganization: (id, body) => request('PUT', `api/organizations/${encodeURIComponent(id)}`, body),
    deleteOrganization: (id) => request('DELETE', `api/organizations/${encodeURIComponent(id)}`),

    listCoordinates: () => request('GET', 'api/coordinates'),
    deleteCoordinates: (id) => request('DELETE', `api/coordinates/${encodeURIComponent(id)}`),
    listAddresses: () => request('GET', 'api/addresses'),
    deleteAddress: (id) => request('DELETE', `api/addresses/${encodeURIComponent(id)}`),

    countByType: (type) => request('GET', `api/special/count-by-type${query({type})}`),
    // fullName может быть пустой строкой, поэтому параметр передаётся всегда
    countFullNameGreater: (fullName) =>
        request('GET', `api/special/count-full-name-greater?fullName=${encodeURIComponent(fullName)}`),
    typeLessThan: (type) => request('GET', `api/special/type-less-than${query({type})}`),
    addEmployee: (id) => request('POST', `api/special/organizations/${encodeURIComponent(id)}/employees`),
    absorb: (absorberId, absorbedId) => request('POST', 'api/special/absorb', {absorberId, absorbedId}),
};
```

Листинг A.51 — src/main/resources/static/js/dialogs.js

```
// Модальные окна: изменение организации и подтверждение удаления.

import {api} from './api.js';
import {clear, h} from './dom.js';
import {toast} from './layout.js';
import {OrganizationForm} from './organization-form.js';
import {onChange} from './realtime.js';

/** @returns Promise<boolean> — подтвердил ли пользователь действие */
export function confirmDialog({title, message, confirmText = 'Подтвердить', danger = false}) {
    return new Promise((resolve) => {
        const cancel = h('button', {type: 'button', class: 'btn', onclick: () => dialog.close('cancel')},
'Отмена');
        const dialog = h('dialog', {class: 'modal modal--small'},
            h('div', {class: 'modal__head'}, h('h2', {}, title)),
            h('div', {class: 'modal__body'},
                [message].flat().map(line => h('p', {}, line)),
                h('div', {class: 'form-actions'},
                    cancel,
                    h('button', {
                        type: 'button',
                        class: `btn ${danger ? 'btn--danger' : 'btn--primary'}`,
                        onclick: () => dialog.close('confirm'),
                    }, confirmText))));
        dialog.addEventListener('close', () => {
            resolve(dialog.returnValue === 'confirm');
            dialog.remove();
        });
        document.body.append(dialog);
        dialog.showModal();
        cancel.focus();
    });
}

/**
 * Отдельное окно изменения организации.
 * @returns Promise с сохранённой организацией или null, если окно закрыли без сохранения
 */
export function openEditDialog(organizationId) {
    const id = Number(organizationId);
    return new Promise((resolve) => {
        let saved = null;
        let saving = false;

        const notice = h('div', {class: 'alert alert--warning', role: 'status'});
        const body = h('div', {class: 'modal__body'}, h('p', {class: 'muted'}, 'Загрузка…'));
        const dialog = h('dialog', {class: 'modal'},
            h('div', {class: 'modal__head'},
                h('h2', {}, `Изменение организации #${id}`),
                h('button', {type: 'button', class: 'icon-btn', 'aria-label': 'Закрыть', onclick: () =>
dialog.close()}, '×')),
            body);

        const form = new OrganizationForm({
            submitLabel: 'Сохранить изменения',
            onCancel: () => dialog.close(),
            onSubmit: async (request) => {
                saving = true;
                try {
                    saved = await api.updateOrganization(id, request);
                    toast(`Организация #${id} сохранена`, 'success');
                    dialog.close();
                } finally {
                    saving = false;
                }
            },
        });

        const load = async () => {
            clear(notice);
            form.setSubmitEnabled(true);
            try {
                const organization = await api.getOrganization(id);
                await form.init(organization);
                clear(body, notice, form.element);
            } catch (e) {
```

```
                clear(body,
                    h('div', {class: 'alert alert--error'}, e.message),
                    h('div', {class: 'form-actions'},
                        h('button', {type: 'button', class: 'btn', onclick: () => dialog.close()}, 'Закрыть')));
            }
        };

        // Пока окно открыто, объект могут изменить или удалить другие пользователи
        const unsubscribe = onChange((event) => {
            if (saving || event.entity !== 'ORGANIZATION' || !event.ids.includes(id)) {
                return;
            }
            if (event.change === 'DELETED') {
                clear(notice, 'Эту организацию только что удалил другой пользователь — сохранить изменения не
получится.');
                form.setSubmitEnabled(false);
            } else if (event.change === 'UPDATED') {
                clear(notice, 'Другой пользователь только что изменил эту организацию. ',
                    h('button', {type: 'button', class: 'btn btn--sm', onclick: load}, 'Загрузить актуальные
данные'));
            }
        });

        dialog.addEventListener('close', () => {
            unsubscribe();
            form.destroy();
            dialog.remove();
            resolve(saved);
        });
        document.body.append(dialog);
        dialog.showModal();
        load();
    });
}

/** @returns Promise<boolean> — была ли организация удалена */
export async function deleteOrganizationWithConfirm(organization) {
    const confirmed = await confirmDialog({
        title: 'Удаление организации',
        message: [
            `Организация #${organization.id} «${organization.name}» будет удалена без возможности
восстановления.`,
            'Связанные с ней координаты и адреса тоже будут удалены, если их больше не использует ни одна
организация.',
        ],
        confirmText: 'Удалить',
        danger: true,
    });
    if (!confirmed) {
        return false;
    }
    try {
        await api.deleteOrganization(organization.id);
        toast(`Организация #${organization.id} удалена`, 'success');
        return true;
    } catch (e) {
        toast(e.message, 'error', 6000);
        return false;
    }
}
```

Листинг A.52 — src/main/resources/static/js/dom.js

```
// Небольшие помощники для построения DOM без innerHTML (защита от XSS в пользовательских строках).

export function h(tag, props = {}, ...children) {
    const el = document.createElement(tag);
    for (const [key, value] of Object.entries(props || {})) {
        if (value === undefined || value === null || value === false) {
            continue;
        }
        if (key === 'class') {
            el.className = value;
        } else if (key === 'dataset') {
            Object.assign(el.dataset, value);
        } else if (key.startsWith('on') && typeof value === 'function') {
            el.addEventListener(key.slice(2).toLowerCase(), value);
        } else if (key === 'value' || key === 'checked' || key === 'disabled' || key === 'selected') {
```

```
            el[key] = value;
        } else {
            el.setAttribute(key, value === true ? '' : value);
        }
    }
    append(el, children);
    return el;
}

function append(el, children) {
    for (const child of children.flat(Infinity)) {
        if (child === undefined || child === null || child === false) {
            continue;
        }
        el.append(child instanceof Node ? child : String(child));
    }
}

export function clear(el, ...children) {
    el.replaceChildren();
    append(el, children);
    return el;
}

export const TYPES = ['GOVERNMENT', 'TRUST', 'PRIVATE_LIMITED_COMPANY'];

export function nullValue() {
    return h('span', {class: 'null'}, '—');
}

export function show(value) {
    return value === null || value === undefined ? nullValue() : String(value);
}

export function typeChip(type) {
    return h('span', {class: `chip chip--${type}`}, type);
}

export function formatDate(iso) {
    if (!iso) {
        return nullValue();
    }
    const date = new Date(iso);
    return Number.isNaN(date.getTime()) ? iso : date.toLocaleString('ru-RU');
}

export function formatCoordinates(c) {
    return c ? `(${c.x}; ${c.y})` : nullValue();
}

export function formatZip(address) {
    if (!address) {
        return nullValue();
    }
    return address.zipCode === '' ? h('span', {class: 'null'}, '«пустой индекс»') : address.zipCode;
}

export function formatString(value) {
    if (value === null || value === undefined) {
        return nullValue();
    }
    return value === '' ? h('span', {class: 'null'}, '«пустая строка»') : value;
}

export function pluralOrganizations(n) {
    const mod10 = n % 10;
    const mod100 = n % 100;
    if (mod10 === 1 && mod100 !== 11) {
        return `${n} организация`;
    }
    if (mod10 >= 2 && mod10 <= 4 && (mod100 < 12 || mod100 > 14)) {
        return `${n} организации`;
    }
    return `${n} организаций`;
}
```

Листинг A.53 — src/main/resources/static/js/layout.js

```
import {h} from './dom.js';

const MENU = [
    {href: 'index.html', title: 'Организации'},
    {href: 'create.html', title: 'Создать'},
    {href: 'view.html', title: 'Поиск по ID'},
    {href: 'special.html', title: 'Спецоперации'},
    {href: 'related.html', title: 'Координаты и адреса'},
];

let toastBox = null;

/** Меню навигации между частями системы и индикатор live-обновлений. */
export function renderLayout(active) {
    const status = h('span', {class: 'live-status', title: 'Синхронизация с другими пользователями'},
'Подключение…');
    document.addEventListener('live-status', (e) => {
        const online = e.detail.online;
        status.classList.toggle('online', online);
        status.classList.toggle('offline', !online);
        status.textContent = online ? 'Онлайн' : 'Нет связи, переподключение…';
    });

    const header = h('header', {class: 'app-header'},
        h('div', {class: 'app-header__inner'},
            h('a', {class: 'brand', href: 'index.html'}, 'ИС «Организации»'),
            h('nav', {class: 'menu'},
                MENU.map(item => h('a', {href: item.href, class: item.href === active ? 'active' : null},
item.title))),
            status));
    document.body.prepend(header);

    toastBox = h('div', {class: 'toasts', 'aria-live': 'polite'});
    document.body.append(toastBox);
}

export function toast(message, type = 'info', timeout = 4000) {
    if (!toastBox) {
        return;
    }
    const el = h('div', {class: `toast toast--${type}`, role: type === 'error' ? 'alert' : 'status'}, message);
    toastBox.append(el);
    setTimeout(() => el.remove(), timeout);
}
```

Листинг A.54 — src/main/resources/static/js/organization-form.js

```
// Форма организации: используется на странице создания и в диалоге изменения.
// Проверки повторяют серверные, чтобы пользователь сразу видел причину ошибки.

import {api, ApiError} from './api.js';
import {clear, h, pluralOrganizations, TYPES} from './dom.js';
import {debounce, onChange} from './realtime.js';

const MAX_LENGTH = 255;
const MAX_X = 549;
const FLOAT_MAX = 3.4028234663852886e38;

let uidCounter = 0;
const uid = () => `of${++uidCounter}`;

// ---------- Разбор и ограничение ввода чисел ----------

export function parseInteger(raw) {
    const s = raw.trim();
    if (s === '') {
        return {empty: true};
    }
    if (!/^[+-]?\d+$/.test(s)) {
        return {error: 'Введите целое число'};
    }
    const value = Number(s);
    if (!Number.isSafeInteger(value)) {
        return {error: `Слишком большое число (максимум ${Number.MAX_SAFE_INTEGER})`};
    }
    return {value};
```

```
}

export function parseDecimal(raw) {
    const s = raw.trim().replace(',', '.');
    if (s === '') {
        return {empty: true};
    }
    if (!/^[+-]?(\d+\.?\d*|\.\d+)([eE][+-]?\d+)?$/.test(s)) {
        return {error: 'Введите число, например 12.5'};
    }
    const value = Number(s);
    if (!Number.isFinite(value)) {
        return {error: 'Слишком большое по модулю число'};
    }
    return {value};
}

/**
 * Текстовое поле, в которое нельзя ввести символы, недопустимые для числа.
 * При попытке ввода такого символа рядом с полем на секунду появляется подсказка.
 */
export function numberInput(kind, onRejected) {
    const disallowed = kind === 'integer' ? /[^\d+-]/g : /[^\d.,eE+-]/g;
    const input = h('input', {
        type: 'text',
        inputmode: kind === 'integer' ? 'numeric' : 'decimal',
        autocomplete: 'off',
        maxlength: 40,
    });
    input.addEventListener('input', () => {
        const {value} = input;
        const cleaned = value.replace(disallowed, '');
        if (cleaned !== value) {
            const caret = value.slice(0, input.selectionStart ?? value.length).replace(disallowed, '').length;
            input.value = cleaned;
            input.setSelectionRange(caret, caret);
            onRejected?.(kind === 'integer'
                ? 'Можно вводить только цифры'
                : 'Можно вводить только цифры, знак, точку или запятую');
        }
    });
    return input;
}

// ---------- Форма ----------

export class OrganizationForm {

    /**
     * @param submitLabel текст кнопки отправки
     * @param onSubmit async (request) => void; ApiError отображается в форме
     * @param onCancel необязательная кнопка "Отмена"
     */
    constructor({submitLabel = 'Сохранить', onSubmit, onCancel = null}) {
        this.onSubmit = onSubmit;
        this.fields = {};
        this.submitted = false;
        this.coordinatesOptions = [];
        this.addressOptions = [];
        this.element = this.build(submitLabel, onCancel);
        this.unsubscribe = onChange(debounce(() => this.loadOptions().catch(() => {
        }), 200));
    }

    async init(organization = null) {
        await this.loadOptions();
        if (organization) {
            this.setValues(organization);
        }
    }

    destroy() {
        this.unsubscribe();
    }

    // ----- построение -----

    build(submitLabel, onCancel) {
```

```
    this.generalError = h('div', {class: 'alert alert--error', role: 'alert'});
    this.inputs = {
        name: h('input', {type: 'text', maxlength: MAX_LENGTH, autocomplete: 'off'}),
        fullName: h('input', {type: 'text', maxlength: MAX_LENGTH, autocomplete: 'off'}),
        type: h('select', {},
            h('option', {value: ''}, '— выберите —'),
            TYPES.map(t => h('option', {value: t}, t))),
        employeesCount: numberInput('integer', msg => this.flashError('employeesCount', msg)),
        annualTurnover: numberInput('integer', msg => this.flashError('annualTurnover', msg)),
        rating: numberInput('decimal', msg => this.flashError('rating', msg)),
    };

    const main = h('div', {class: 'form-grid'},
        this.field('name', 'Название', this.inputs.name,
            {required: true, hint: 'Не может быть пустым, до 255 символов'}),
        this.field('fullName', 'Полное название', this.inputs.fullName,
            {required: true, hint: 'Обязательно, но может быть пустой строкой'}),
        this.field('type', 'Тип', this.inputs.type, {required: true}),
        this.field('employeesCount', 'Количество сотрудников', this.inputs.employeesCount,
            {required: true, hint: 'Целое число больше 0'}),
        this.field('annualTurnover', 'Годовой оборот', this.inputs.annualTurnover,
            {hint: 'Необязательно. Целое число больше 0'}),
        this.field('rating', 'Рейтинг', this.inputs.rating,
            {hint: 'Необязательно. Число больше 0'}));

    this.coordinates = this.buildCoordinatesPicker();
    this.officialAddress = this.buildAddressPicker('officialAddress', 'Официальный адрес');
    this.postalAddress = this.buildAddressPicker('postalAddress', 'Почтовый адрес');

    this.submitButton = h('button', {type: 'submit', class: 'btn btn--primary'}, submitLabel);
    const actions = h('div', {class: 'form-actions'},
        onCancel ? h('button', {type: 'button', class: 'btn', onclick: onCancel}, 'Отмена') : null,
        this.submitButton);

    const form = h('form', {novalidate: true, autocomplete: 'off'},
        this.generalError,
        h('div', {class: 'form-section-title'}, 'Основные данные'),
        main,
        h('div', {class: 'form-section-title'}, 'Связанные объекты'),
        h('div', {class: 'relations-grid'},
            this.coordinates.element, this.officialAddress.element, this.postalAddress.element),
        actions);

    form.addEventListener('submit', (e) => {
        e.preventDefault();
        this.submit();
    });
    // После первой попытки отправки ошибки пересчитываются на лету
    const revalidate = () => {
        if (this.submitted) {
            this.showErrors(this.collect().errors);
        }
    };
    form.addEventListener('input', revalidate);
    form.addEventListener('change', revalidate);
    return form;
}

field(key, label, control, {required = false, hint = null} = {}) {
    const id = `${uid()}-${key.replace('.', '-')}`;
    control.id = id;
    const error = h('div', {class: 'error-text', id: `${id}-error`});
    control.setAttribute('aria-describedby', error.id);
    const wrap = h('div', {class: 'field'},
        h('label', {for: id}, label, required ? h('span', {class: 'req'}, ' *') : null),
        control,
        hint ? h('div', {class: 'hint'}, hint) : null,
        error);
    this.fields[key] = {wrap, error, control};
    return wrap;
}

/** Переключатель режима: несколько радиокнопок в виде сегментированного контрола. */
segmented(options) {
    const name = uid();
    const radios = {};
    const el = h('div', {class: 'segmented', role: 'radiogroup'},
        options.map(([value, label]) => {
```

```
                radios[value] = h('input', {type: 'radio', name, value});
                return h('label', {}, radios[value], label);
            }));
        return {el, radios};
    }

    buildCoordinatesPicker() {
        const {el: switcher, radios} = this.segmented([['new', 'Новые'], ['existing', 'Существующие']]);
        const x = numberInput('decimal', msg => this.flashError('coordinates.x', msg));
        const y = numberInput('decimal', msg => this.flashError('coordinates.y', msg));
        const select = h('select', {});
        const selectId = uid();
        select.id = selectId;

        const newBox = h('div', {class: 'form-grid'},
            this.field('coordinates.x', 'X', x, {required: true, hint: 'Не больше 549'}),
            this.field('coordinates.y', 'Y', y, {required: true}));
        const existingBox = h('div', {class: 'field'},
            h('label', {for: selectId}, 'Выберите из уже созданных'),
            select);
        const error = h('div', {class: 'error-text'});
        const element = h('fieldset', {class: 'group'},
            h('legend', {}, 'Координаты', h('span', {class: 'req'}, ' *')),
            switcher, newBox, existingBox, error);
        this.fields.coordinates = {wrap: element, error, control: select};

        const picker = {element, radios, x, y, select, mode: () => (radios.existing.checked ? 'existing' :
'new')};
        picker.sync = () => {
            newBox.hidden = picker.mode() !== 'new';
            existingBox.hidden = picker.mode() !== 'existing';
        };
        radios.new.addEventListener('change', () => {
            // Удобно поправить значения выбранных координат: получатся новые координаты
            const selected = this.coordinatesOptions.find(c => String(c.id) === select.value);
            if (selected && x.value === '' && y.value === '') {
                x.value = String(selected.x);
                y.value = String(selected.y);
            }
            picker.sync();
        });
        radios.existing.addEventListener('change', picker.sync);
        radios.new.checked = true;
        picker.sync();
        return picker;
    }

    buildAddressPicker(key, title) {
        const {el: switcher, radios} = this.segmented([['none', 'Нет'], ['new', 'Новый'], ['existing',
'Существующий']]);
        const zip = h('input', {type: 'text', maxlength: MAX_LENGTH, autocomplete: 'off'});
        const select = h('select', {});
        const selectId = uid();
        select.id = selectId;

        const newBox = this.field(`${key}.zipCode`, 'Почтовый индекс (zipCode)', zip,
            {required: true, hint: 'Может быть пустой строкой, до 255 символов'});
        const existingBox = h('div', {class: 'field'},
            h('label', {for: selectId}, 'Выберите из уже созданных'),
            select);
        const error = h('div', {class: 'error-text'});
        const element = h('fieldset', {class: 'group'},
            h('legend', {}, title),
            switcher, newBox, existingBox, error);
        this.fields[key] = {wrap: element, error, control: select};

        const picker = {
            key, element, radios, zip, select,
            mode: () => Object.keys(radios).find(m => radios[m].checked),
        };
        picker.sync = () => {
            newBox.hidden = picker.mode() !== 'new';
            existingBox.hidden = picker.mode() !== 'existing';
        };
        Object.values(radios).forEach(r => r.addEventListener('change', picker.sync));
        radios.none.checked = true;
        picker.sync();
        return picker;
```

```
    }

    // ----- список существующих вспомогательных объектов -----

    async loadOptions() {
        const [coordinates, addresses] = await Promise.all([api.listCoordinates(), api.listAddresses()]);
        this.coordinatesOptions = coordinates;
        this.addressOptions = addresses;

        this.fillSelect('coordinates', this.coordinates, coordinates,
            c => `#${c.id}: (${c.x}; ${c.y}) — ${pluralOrganizations(c.usages)}`,
            id => `Выбранные координаты #${id} удалены другим пользователем, выберите другие`);
        for (const picker of [this.officialAddress, this.postalAddress]) {
            this.fillSelect(picker.key, picker, addresses,
                a => `#${a.id}: ${a.zipCode === '' ? '«пустой индекс»' : a.zipCode} —
${pluralOrganizations(a.usages)}`,
                id => `Выбранный адрес #${id} удалён другим пользователем, выберите другой`);
        }
    }

    fillSelect(key, picker, items, label, goneMessage) {
        const {select, radios} = picker;
        const current = select.value;
        clear(select,
            h('option', {value: ''}, items.length ? '— выберите —' : '— пока нет ни одного —'),
            items.map(item => h('option', {value: String(item.id)}, label(item))));
        if (current && items.some(item => String(item.id) === current)) {
            select.value = current;
        } else if (current && picker.mode() === 'existing') {
            this.setFieldError(key, goneMessage(current));
        }
        radios.existing.disabled = items.length === 0 && picker.mode() !== 'existing';
    }

    // ----- значения -----

    setValues(o) {
        this.inputs.name.value = o.name;
        this.inputs.fullName.value = o.fullName;
        this.inputs.type.value = o.type;
        this.inputs.employeesCount.value = String(o.employeesCount);
        this.inputs.annualTurnover.value = o.annualTurnover ?? '';
        this.inputs.rating.value = o.rating ?? '';

        const c = this.coordinates;
        c.radios.existing.disabled = false;
        c.radios.existing.checked = true;
        c.select.value = String(o.coordinates.id);
        c.x.value = String(o.coordinates.x);
        c.y.value = String(o.coordinates.y);
        c.sync();

        for (const [picker, address] of [[this.officialAddress, o.officialAddress], [this.postalAddress,
o.postalAddress]]) {
            picker.zip.value = address ? address.zipCode : '';
            if (address) {
                picker.radios.existing.disabled = false;
                picker.radios.existing.checked = true;
                picker.select.value = String(address.id);
            } else {
                picker.radios.none.checked = true;
            }
            picker.sync();
        }
        this.submitted = false;
        this.showErrors({});
    }

    reset() {
        this.element.reset();
        this.coordinates.radios.new.checked = true;
        this.coordinates.sync();
        for (const picker of [this.officialAddress, this.postalAddress]) {
            picker.radios.none.checked = true;
            picker.sync();
        }
        this.submitted = false;
        this.showErrors({});
```

```
}

/** Собирает запрос и проверяет его так же, как сервер. */
collect() {
    const errors = {};
    const i = this.inputs;

    const name = i.name.value;
    if (name.trim() === '') {
        errors.name = 'Название не может быть пустым';
    }
    const type = i.type.value;
    if (!type) {
        errors.type = 'Выберите тип организации';
    }

    const request = {
        name,
        coordinates: this.collectCoordinates(errors),
        officialAddress: this.collectAddress(this.officialAddress, errors),
        annualTurnover: this.readPositiveInteger('annualTurnover', null,
            'Годовой оборот должен быть больше 0', errors),
        employeesCount: this.readPositiveInteger('employeesCount', 'Укажите количество сотрудников',
            'Количество сотрудников должно быть больше 0', errors),
        rating: this.readRating(errors),
        fullName: i.fullName.value,
        type: type || null,
        postalAddress: this.collectAddress(this.postalAddress, errors),
    };
    return {request, errors};
}

/** @param requiredMessage null — поле необязательное */
readPositiveInteger(key, requiredMessage, positiveMessage, errors) {
    const parsed = parseInteger(this.inputs[key].value);
    if (parsed.empty) {
        if (requiredMessage) {
            errors[key] = requiredMessage;
        }
        return null;
    }
    if (parsed.error) {
        errors[key] = parsed.error;
        return null;
    }
    if (parsed.value <= 0) {
        errors[key] = positiveMessage;
        return null;
    }
    return parsed.value;
}

readRating(errors) {
    const parsed = parseDecimal(this.inputs.rating.value);
    if (parsed.empty) {
        return null;
    }
    if (parsed.error) {
        errors.rating = parsed.error;
        return null;
    }
    // Рейтинг хранится как float: слишком маленькое значение превратится в 0
    if (Math.fround(parsed.value) <= 0) {
        errors.rating = 'Рейтинг должен быть больше 0';
        return null;
    }
    if (parsed.value > FLOAT_MAX) {
        errors.rating = 'Рейтинг слишком большой (максимум ≈ 3.4e38)';
        return null;
    }
    return parsed.value;
}

collectCoordinates(errors) {
    const c = this.coordinates;
    if (c.mode() === 'existing') {
        if (!c.select.value) {
            errors.coordinates = 'Выберите координаты из списка';
```

```
            return null;
        }
        return {id: Number(c.select.value)};
    }
    const x = parseDecimal(c.x.value);
    const y = parseDecimal(c.y.value);
    if (x.empty) {
        errors['coordinates.x'] = 'Укажите X';
    } else if (x.error) {
        errors['coordinates.x'] = x.error;
    } else if (x.value > MAX_X) {
        errors['coordinates.x'] = 'X не может быть больше 549';
    }
    if (y.empty) {
        errors['coordinates.y'] = 'Укажите Y';
    } else if (y.error) {
        errors['coordinates.y'] = y.error;
    }
    return {x: x.value ?? null, y: y.value ?? null};
}

collectAddress(picker, errors) {
    switch (picker.mode()) {
        case 'new':
            return {zipCode: picker.zip.value};
        case 'existing':
            if (!picker.select.value) {
                errors[picker.key] = 'Выберите адрес из списка';
                return null;
            }
            return {id: Number(picker.select.value)};
        default:
            return null;
    }
}

// ----- ошибки -----

showErrors(fieldErrors, generalMessage = null) {
    const unknown = [];
    for (const [key, field] of Object.entries(this.fields)) {
        const message = fieldErrors[key] ?? null;
        field.error.textContent = message ?? '';
        field.wrap.classList.toggle('invalid', Boolean(message));
    }
    for (const [key, message] of Object.entries(fieldErrors)) {
        if (!this.fields[this.fieldKey(key)]) {
            unknown.push(message);
        } else if (!this.fields[key]) {
            this.setFieldError(this.fieldKey(key), message);
        }
    }
    if (generalMessage || unknown.length) {
        clear(this.generalError,
            generalMessage,
            unknown.length ? h('ul', {}, unknown.map(m => h('li', {}, m))) : null);
    } else {
        clear(this.generalError);
    }
}

/** "coordinates.id" -> "coordinates": такие ошибки показываются у группы полей. */
fieldKey(key) {
    return this.fields[key] ? key : key.split('.')[0];
}

setFieldError(key, message) {
    const field = this.fields[key];
    if (field) {
        field.error.textContent = message;
        field.wrap.classList.add('invalid');
    }
}

flashError(key, message) {
    const field = this.fields[key];
    if (!field || field.error.textContent) {
        return;
```

```
        }
        field.error.textContent = message;
        setTimeout(() => {
            if (field.error.textContent === message) {
                field.error.textContent = '';
            }
        }, 1800);
    }

    focusFirstInvalid() {
        const invalid = Object.values(this.fields).find(f => f.wrap.classList.contains('invalid'));
        const target = invalid?.wrap.querySelector('input:not([type=radio]), select');
        (target ?? invalid?.wrap)?.focus?.();
        invalid?.wrap.scrollIntoView({block: 'center', behavior: 'smooth'});
    }

    setSubmitEnabled(enabled) {
        this.blocked = !enabled;
        this.submitButton.disabled = this.blocked;
    }

    // ----- отправка -----

    async submit() {
        this.submitted = true;
        const {request, errors} = this.collect();
        if (Object.keys(errors).length) {
            this.showErrors(errors, 'Исправьте ошибки в выделенных полях');
            this.focusFirstInvalid();
            return;
        }
        this.showErrors({});
        this.submitButton.disabled = true;
        try {
            await this.onSubmit(request);
        } catch (e) {
            if (e instanceof ApiError) {
                this.showErrors(e.fieldErrors, e.message);
                this.focusFirstInvalid();
            } else {
                this.showErrors({}, 'Непредвиденная ошибка: ' + e.message);
                console.error(e);
            }
        } finally {
            this.submitButton.disabled = Boolean(this.blocked);
        }
    }
}
```

Листинг A.55 — src/main/resources/static/js/realtime.js

```
// Подписка на изменения, которые сервер рассылает по WebSocket после каждой транзакции.
// Событие: {entity: 'ORGANIZATION'|'COORDINATES'|'ADDRESS', change: 'CREATED'|'UPDATED'|'DELETED', ids: [...]}.
// После переподключения приходит {entity: '*', change: 'RESYNC'}: пока связи не было, события могли потеряться.

const listeners = new Set();
let socket = null;
let attempt = 0;
let everConnected = false;

export function onChange(listener) {
    listeners.add(listener);
    connect();
    return () => listeners.delete(listener);
}

function emit(event) {
    for (const listener of [...listeners]) {
        try {
            listener(event);
        } catch (e) {
            console.error('Ошибка обработчика обновлений', e);
        }
    }
}

function setStatus(online) {
    document.dispatchEvent(new CustomEvent('live-status', {detail: {online}}));
```

```
}

function connect() {
    if (socket) {
        return;
    }
    const url = new URL('ws/updates', location.href);
    url.protocol = location.protocol === 'https:' ? 'wss:' : 'ws:';
    socket = new WebSocket(url);

    socket.addEventListener('open', () => {
        setStatus(true);
        if (everConnected) {
            emit({entity: '*', change: 'RESYNC', ids: []});
        }
        everConnected = true;
        attempt = 0;
    });
    socket.addEventListener('message', (message) => {
        try {
            emit(JSON.parse(message.data));
        } catch (e) {
            console.error('Некорректное сообщение об изменении', e);
        }
    });
    socket.addEventListener('close', () => {
        setStatus(false);
        socket = null;
        const delay = Math.min(10_000, 500 * 2 ** attempt++);
        setTimeout(connect, delay);
    });
}

/** Склеивает пачку событий в один вызов (например, удаление адреса удаляет несколько организаций). */
export function debounce(fn, ms = 150) {
    let timer = null;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), ms);
    };
}
```

Листинг A.56 — src/main/resources/static/js/pages/create.js

```
import {api} from '../api.js';
import {clear, h} from '../dom.js';
import {renderLayout, toast} from '../layout.js';
import {OrganizationForm} from '../organization-form.js';

renderLayout('create.html');

const app = document.getElementById('app');
const success = h('div', {class: 'alert alert--success', role: 'status'});

const form = new OrganizationForm({
    submitLabel: 'Создать организацию',
    onSubmit: async (request) => {
        const created = await api.createOrganization(request);
        form.reset();
        clear(success,
            `Организация #${created.id} «${created.name}» создана. `,
            h('a', {href: `view.html?id=${created.id}`}, 'Открыть'),
            ' · ',
            h('a', {href: 'index.html'}, 'Перейти к списку'));
        toast(`Организация #${created.id} создана`, 'success');
        window.scrollTo({top: 0, behavior: 'smooth'});
    },
});

app.append(success, form.element);
form.init().catch((e) => form.showErrors({}, `Не удалось загрузить существующие координаты и адреса:
${e.message}`));
```

Листинг A.57 — src/main/resources/static/js/pages/list.js

```
import {api} from '../api.js';
import {deleteOrganizationWithConfirm, openEditDialog} from '../dialogs.js';
import {clear, formatCoordinates, formatDate, formatString, formatZip, h, show, TYPES, typeChip} from
```

```
'../dom.js';
import {renderLayout} from '../layout.js';
import {debounce, onChange} from '../realtime.js';

renderLayout('index.html');

const COLUMNS = [
    {key: 'id', title: 'ID', cls: 'num', render: o => o.id},
    {key: 'name', title: 'Название', render: o => o.name},
    {key: 'coordinates', title: 'Координаты (x; y)', cls: 'nowrap', render: o =>
formatCoordinates(o.coordinates)},
    {key: 'creationDate', title: 'Дата создания', render: o => formatDate(o.creationDate)},
    {key: 'officialAddress', title: 'Офиц. адрес (индекс)', render: o => formatZip(o.officialAddress)},
    {key: 'annualTurnover', title: 'Годовой оборот', cls: 'num', render: o => show(o.annualTurnover)},
    {key: 'employeesCount', title: 'Сотрудники', cls: 'num', render: o => o.employeesCount},
    {key: 'rating', title: 'Рейтинг', cls: 'num', render: o => show(o.rating)},
    {key: 'fullName', title: 'Полное название', render: o => formatString(o.fullName)},
    {key: 'type', title: 'Тип', render: o => typeChip(o.type)},
    {key: 'postalAddress', title: 'Почт. адрес (индекс)', render: o => formatZip(o.postalAddress)},
];

// Фильтры по строковым колонкам, полное совпадение.
// canBeEmpty — значение поля может быть пустой строкой, и по ней тоже можно отфильтровать.
const FILTERS = [
    {key: 'name', label: 'Название'},
    {key: 'fullName', label: 'Полное название', canBeEmpty: true},
    {key: 'type', label: 'Тип', options: TYPES},
    {key: 'officialZipCode', label: 'Индекс офиц. адреса', canBeEmpty: true},
    {key: 'postalZipCode', label: 'Индекс почт. адреса', canBeEmpty: true},
];

const PAGE_SIZES = [5, 10, 20, 50];

// ---------- состояние (хранится в адресной строке) ----------

const state = readState();

function readState() {
    const params = new URLSearchParams(location.search);
    const filters = {};
    for (const f of FILTERS) {
        const value = params.get(f.key);
        if (value || (value === '' && f.canBeEmpty)) {
            filters[f.key] = value;
        }
    }
    const size = Number(params.get('size'));
    const sort = params.get('sort');
    return {
        page: Math.max(0, Number.parseInt(params.get('page'), 10) || 0),
        size: PAGE_SIZES.includes(size) ? size : 10,
        sort: COLUMNS.some(c => c.key === sort) ? sort : 'id',
        direction: params.get('direction') === 'desc' ? 'desc' : 'asc',
        filters,
    };
}

function writeState() {
    const params = new URLSearchParams({
        page: state.page, size: state.size, sort: state.sort, direction: state.direction, ...state.filters,
    });
    history.replaceState(null, '', `?${params}`);
}

// ---------- разметка ----------

const app = document.getElementById('app');
const filterInputs = {};
const emptyChecks = {};

const filtersForm = h('form', {class: 'card filters', autocomplete: 'off'},
    FILTERS.map(f => {
        const input = f.options
            ? h('select', {}, h('option', {value: ''}, 'Любой'), f.options.map(o => h('option', {value: o}, o)))
            : h('input', {type: 'text', placeholder: 'Точное значение', maxlength: 255});
        input.value = state.filters[f.key] ?? '';
        filterInputs[f.key] = input;
        if (!f.canBeEmpty) {
```

```
            return h('label', {class: 'field'}, h('span', {class: 'field-label'}, f.label), input);
        }
        // Пустое поле ввода означает «без фильтра», поэтому пустая строка выбирается флажком
        const empty = h('input', {
            type: 'checkbox',
            checked: state.filters[f.key] === '',
            onchange: () => {
                input.disabled = empty.checked;
                if (empty.checked) {
                    input.value = '';
                }
            },
        });
        input.disabled = empty.checked;
        emptyChecks[f.key] = empty;
        return h('div', {class: 'field'},
            h('label', {class: 'field'}, h('span', {class: 'field-label'}, f.label), input),
            h('label', {class: 'hint check'}, empty, ' пустая строка'));
    }),
    h('div', {class: 'btn-row'},
        h('span', {class: 'hint'}, 'Строка показывается, только если значение совпадает полностью'),
        h('button', {type: 'button', class: 'btn', onclick: resetFilters}, 'Сбросить'),
        h('button', {type: 'submit', class: 'btn btn--primary'}, 'Применить')));
filtersForm.addEventListener('submit', (e) => {
    e.preventDefault();
    applyFilters();
});

const summary = h('div', {class: 'muted'});
const pager = h('div', {class: 'pager'});
const errorBox = h('div', {class: 'alert alert--error', role: 'alert'});
const thead = h('thead');
const tbody = h('tbody');

app.append(
    filtersForm,
    h('div', {class: 'toolbar'}, summary, pager),
    errorBox,
    h('div', {class: 'table-wrap'}, h('table', {class: 'data'}, thead, tbody)));

function renderHead() {
    clear(thead, h('tr', {},
        COLUMNS.map(c => {
            const sorted = state.sort === c.key;
            const mark = sorted ? (state.direction === 'asc' ? '▲' : '▼') : '↕';
            return h('th', {
                    class: `sortable${sorted ? ' sorted' : ''}${c.cls === 'num' ? ' num' : ''}`,
                    tabindex: 0,
                    title: 'Нажмите, чтобы отсортировать',
                    'aria-sort': sorted ? (state.direction === 'asc' ? 'ascending' : 'descending') : null,
                    onclick: () => toggleSort(c.key),
                    onkeydown: (e) => {
                        if (e.key === 'Enter' || e.key === ' ') {
                            e.preventDefault();
                            toggleSort(c.key);
                        }
                    },
                },
                c.title, h('span', {class: 'sort-mark'}, mark));
        }),
        h('th', {class: 'actions'}, 'Действия')));
}

function renderRows(rows, highlight) {
    if (!rows.length) {
        const filtered = Object.keys(state.filters).length > 0;
        clear(tbody, h('tr', {}, h('td', {class: 'empty', colspan: COLUMNS.length + 1},
            filtered ? 'По заданным фильтрам ничего не найдено' : 'Организаций пока нет. ',
            filtered ? null : h('a', {href: 'create.html'}, 'Создать первую'))));
        return;
    }
    clear(tbody, rows.map(o => h('tr', {class: highlight.has(o.id) ? 'flash' : null},
        COLUMNS.map(c => h('td', {class: c.cls}, c.render(o))),
        h('td', {class: 'actions'},
            h('a', {class: 'btn btn--sm btn--ghost', href: `view.html?id=${o.id}`}, 'Открыть'),
            ' ',
            h('button', {type: 'button', class: 'btn btn--sm', onclick: () => edit(o.id)}, 'Изменить'),
            ' ',
```

```
            h('button', {type: 'button', class: 'btn btn--sm btn--danger', onclick: () => remove(o)},
'Удалить')))));
}

function renderPager(data) {
    const total = data.totalElements;
    const from = total === 0 ? 0 : data.page * data.size + 1;
    const to = data.page * data.size + data.content.length;
    summary.textContent = total === 0 ? 'Записей нет' : `Показаны ${from}–${to} из ${total}`;

    const pages = Math.max(1, data.totalPages);
    const go = (page) => () => {
        state.page = page;
        refresh();
    };
    const sizeSelect = h('select', {
        'aria-label': 'Строк на странице',
        onchange: (e) => {
            state.size = Number(e.target.value);
            state.page = 0;
            refresh();
        },
    }, PAGE_SIZES.map(s => h('option', {value: String(s)}, `${s} на странице`)));
    sizeSelect.value = String(state.size);

    clear(pager,
        h('button', {type: 'button', class: 'btn btn--sm', disabled: data.page === 0, onclick: go(0), title:
'Первая'}, '«'),
        h('button', {type: 'button', class: 'btn btn--sm', disabled: data.page === 0, onclick: go(data.page -
1)}, '‹ Назад'),
        h('span', {class: 'page-info'}, `Страница ${data.page + 1} из ${pages}`),
        h('button', {type: 'button', class: 'btn btn--sm', disabled: data.page + 1 >= pages, onclick:
go(data.page + 1)}, 'Вперёд ›'),
        h('button', {type: 'button', class: 'btn btn--sm', disabled: data.page + 1 >= pages, onclick: go(pages -
1), title: 'Последняя'}, '»'),
        sizeSelect);
}

// ---------- действия ----------

let requestCounter = 0;

async function load(highlightIds = []) {
    const current = ++requestCounter;
    try {
        const data = await api.listOrganizations({
            page: state.page, size: state.size, sort: state.sort, direction: state.direction, ...state.filters,
        });
        if (current !== requestCounter) {
            return;
        }
        // Если последнюю страницу опустошили (например, удаление у другого пользователя) — шаг назад
        if (data.totalPages > 0 && state.page >= data.totalPages) {
            state.page = data.totalPages - 1;
            writeState();
            return load(highlightIds);
        }
        clear(errorBox);
        renderRows(data.content, new Set(highlightIds));
        renderPager(data);
    } catch (e) {
        if (current === requestCounter) {
            clear(errorBox, e.message);
        }
    }
}

function refresh() {
    writeState();
    renderHead();
    return load();
}

function toggleSort(key) {
    if (state.sort === key) {
        state.direction = state.direction === 'asc' ? 'desc' : 'asc';
    } else {
        state.sort = key;
```

```
        state.direction = 'asc';
    }
    state.page = 0;
    refresh();
}

function applyFilters() {
    state.filters = {};
    for (const [key, input] of Object.entries(filterInputs)) {
        if (emptyChecks[key]?.checked) {
            state.filters[key] = '';
        } else if (input.value !== '') {
            state.filters[key] = input.value;
        }
    }
    state.page = 0;
    refresh();
}

function resetFilters() {
    Object.values(filterInputs).forEach(input => {
        input.value = '';
        input.disabled = false;
    });
    Object.values(emptyChecks).forEach(check => {
        check.checked = false;
    });
    applyFilters();
}

async function edit(id) {
    if (await openEditDialog(id)) {
        load([id]);
    }
}

async function remove(organization) {
    if (await deleteOrganizationWithConfirm(organization)) {
        load();
    }
}

// ---------- синхронизация с другими пользователями ----------

const changedIds = new Set();
const reloadAfterChanges = debounce(() => {
    const ids = [...changedIds];
    changedIds.clear();
    load(ids);
});

onChange((event) => {
    if (event.entity === 'ORGANIZATION' || event.entity === '*') {
        if (event.change === 'CREATED' || event.change === 'UPDATED') {
            event.ids.forEach(id => changedIds.add(id));
        }
        reloadAfterChanges();
    }
});

refresh();
```

Листинг A.58 — src/main/resources/static/js/pages/related.js

```
import {api} from '../api.js';
import {confirmDialog} from '../dialogs.js';
import {clear, formatString, h, pluralOrganizations} from '../dom.js';
import {renderLayout, toast} from '../layout.js';
import {debounce, onChange} from '../realtime.js';

renderLayout('related.html');

const app = document.getElementById('app');

function section(title) {
    const body = h('div', {}, h('p', {class: 'muted'}, 'Загрузка…'));
    app.append(h('section', {class: 'card'}, h('h2', {}, title), body));
    return body;
```

```
}

const coordinatesBox = section('Координаты');
const addressesBox = section('Адреса');

function table(headers, rows, emptyText) {
    if (!rows.length) {
        return h('div', {class: 'muted'}, emptyText);
    }
    return h('div', {class: 'table-wrap'}, h('table', {class: 'data'},
        h('thead', {}, h('tr', {},
            headers.map(t => h('th', {class: t === 'ID' || t === 'X' || t === 'Y' ? 'num' : null}, t)),
            h('th', {}, ''))),
        h('tbody', {}, rows)));
}

async function remove(kind, item, describe) {
    const confirmed = await confirmDialog({
        title: `Удаление: ${describe}`,
        message: item.usages > 0
            ? [`Вместе с объектом будут удалены ${pluralOrganizations(item.usages)}, которые на него
ссылаются.`,
                'Действие нельзя отменить.']
            : 'Объект не используется организациями. Действие нельзя отменить.',
        confirmText: 'Удалить',
        danger: true,
    });
    if (!confirmed) {
        return;
    }
    try {
        if (kind === 'coordinates') {
            await api.deleteCoordinates(item.id);
        } else {
            await api.deleteAddress(item.id);
        }
        toast(`Удалено: ${describe}`, 'success');
        load();
    } catch (e) {
        toast(e.message, 'error', 6000);
    }
}

async function load() {
    try {
        const [coordinates, addresses] = await Promise.all([api.listCoordinates(), api.listAddresses()]);
        clear(coordinatesBox, table(['ID', 'X', 'Y', 'Используется'],
            coordinates.map(c => h('tr', {},
                h('td', {class: 'num'}, c.id),
                h('td', {class: 'num'}, c.x),
                h('td', {class: 'num'}, c.y),
                h('td', {}, pluralOrganizations(c.usages)),
                h('td', {class: 'actions'}, h('button', {
                    type: 'button', class: 'btn btn--sm btn--danger',
                    onclick: () => remove('coordinates', c, `координаты #${c.id}`),
                }, 'Удалить')))),
            'Координат пока нет'));
        clear(addressesBox, table(['ID', 'Индекс (zipCode)', 'Используется'],
            addresses.map(a => h('tr', {},
                h('td', {class: 'num'}, a.id),
                h('td', {}, formatString(a.zipCode)),
                h('td', {}, pluralOrganizations(a.usages)),
                h('td', {class: 'actions'}, h('button', {
                    type: 'button', class: 'btn btn--sm btn--danger',
                    onclick: () => remove('address', a, `адрес #${a.id}`),
                }, 'Удалить')))),
            'Адресов пока нет'));
    } catch (e) {
        clear(coordinatesBox, h('div', {class: 'alert alert--error'}, e.message));
        clear(addressesBox);
    }
}

onChange(debounce(load));
load();
```

Листинг A.59 — src/main/resources/static/js/pages/special.js

```
import {api, ApiError} from '../api.js';
import {clear, formatString, h, pluralOrganizations, TYPES, typeChip} from '../dom.js';
import {renderLayout, toast} from '../layout.js';
import {numberInput, parseInteger} from '../organization-form.js';
import {debounce, onChange} from '../realtime.js';

renderLayout('special.html');

const app = document.getElementById('app');
let uid = 0;

// ---------- общие помощники ----------

function field(label, control, hint = null) {
    const id = `sp-${++uid}`;
    control.id = id;
    const error = h('div', {class: 'error-text', id: `${id}-error`});
    control.setAttribute('aria-describedby', error.id);
    const wrap = h('div', {class: 'field'}, h('label', {for: id}, label), control,
        hint ? h('div', {class: 'hint'}, hint) : null, error);
    return {wrap, control, error};
}

function setError(f, message) {
    f.error.textContent = message ?? '';
    f.wrap.classList.toggle('invalid', Boolean(message));
}

function typeSelect() {
    return h('select', {}, h('option', {value: ''}, '— выберите —'), TYPES.map(t => h('option', {value: t},
t)));
}

function idInput(f) {
    return numberInput('integer', (msg) => setError(f(), msg));
}

/** Проверка поля ID: возвращает число или null, показывая ошибку у поля. */
function readId(f) {
    const parsed = parseInteger(f.control.value);
    let error = null;
    if (parsed.empty) {
        error = 'Введите ID организации';
    } else if (parsed.error) {
        error = parsed.error;
    } else if (parsed.value <= 0) {
        error = 'ID должен быть больше 0';
    }
    setError(f, error);
    return error ? null : parsed.value;
}

function operationCard(title, description, fields, buttonText, run, wide = false) {
    const result = h('div', {class: 'result', 'aria-live': 'polite'});
    const button = h('button', {type: 'submit', class: 'btn btn--primary'}, buttonText);
    const form = h('form', {class: 'inline-form', novalidate: true}, fields.map(f => f.wrap), button);
    form.addEventListener('submit', async (e) => {
        e.preventDefault();
        button.disabled = true;
        try {
            await run(result);
        } catch (err) {
            showError(result, fields, err);
        } finally {
            button.disabled = false;
        }
    });
    app.append(h('section', {class: wide ? 'card wide' : 'card'},
        h('h2', {}, title), h('p', {class: 'muted'}, description), form, result));
    return result;
}

function showError(result, fields, err) {
    if (err instanceof ApiError) {
        // Ошибки по полям (например, absorbedId) показываем у соответствующих полей
        for (const f of fields) {
```

```
            if (f.name && err.fieldErrors[f.name]) {
                setError(f, err.fieldErrors[f.name]);
            }
        }
        clear(result, h('div', {class: 'alert alert--error'}, err.message));
    } else {
        clear(result, h('div', {class: 'alert alert--error'}, `Непредвиденная ошибка: ${err.message}`));
        console.error(err);
    }
}

function organizationsTable(list) {
    if (!list.length) {
        return h('div', {class: 'alert alert--warning'}, 'Подходящих организаций нет');
    }
    return h('div', {class: 'table-wrap'}, h('table', {class: 'data'},
        h('thead', {}, h('tr', {},
            ['ID', 'Название', 'Полное название', 'Тип', 'Сотрудники'].map(t => h('th', {}, t)))),
        h('tbody', {}, list.map(o => h('tr', {},
            h('td', {class: 'num'}, h('a', {href: `view.html?id=${o.id}`}, o.id)),
            h('td', {}, o.name),
            h('td', {}, formatString(o.fullName)),
            h('td', {}, typeChip(o.type)),
            h('td', {class: 'num'}, o.employeesCount))))));
}

// Запросы 1–3 перевыполняются, когда данные меняют другие пользователи
const liveQueries = [];

// ---------- 1. Количество с заданным type ----------
{
    const typeField = field('Тип', typeSelect());
    let lastType = null;
    const result = operationCard(
        'Количество организаций заданного типа',
        'Возвращает число организаций, у которых поле type равно выбранному значению.',
        [typeField], 'Посчитать',
        async (out) => {
            const type = typeField.control.value;
            setError(typeField, type ? null : 'Выберите тип');
            if (!type) {
                return;
            }
            lastType = type;
            await show(out, type);
        });
    const show = async (out, type) => {
        const {count} = await api.countByType(type);
        clear(out, h('div', {class: 'big-number'}, count), h('div', {class: 'muted'}, `с типом ${type}`));
    };
    liveQueries.push(() => lastType && show(result, lastType));
}

// ---------- 2. Количество с fullName больше заданного ----------
{
    const nameField = field('fullName', h('input', {type: 'text', maxlength: 255, autocomplete: 'off'}),
        'Строки сравниваются в БД; пустое значение тоже допустимо');
    let lastValue = null;
    const result = operationCard(
        'Количество организаций с fullName больше заданного',
        'Возвращает число организаций, у которых полное название лексикографически больше введённой строки.',
        [nameField], 'Посчитать',
        async (out) => {
            lastValue = nameField.control.value;
            await show(out, lastValue);
        });
    const show = async (out, value) => {
        const {count} = await api.countFullNameGreater(value);
        clear(out, h('div', {class: 'big-number'}, count),
            h('div', {class: 'muted'}, value === '' ? 'с fullName больше пустой строки' : `с fullName >
«${value}»`));
    };
    liveQueries.push(() => lastValue !== null && show(result, lastValue));
}

// ---------- 3. Организации с type меньше заданного ----------
{
    const typeField = field('Тип', typeSelect(), `Порядок: ${TYPES.join(' < ')}`);
```

```
    let lastType = null;
    const result = operationCard(
        'Организации с типом меньше заданного',
        'Возвращает массив организаций, у которых type меньше выбранного (в порядке объявления перечисления).',
        [typeField], 'Найти',
        async (out) => {
            const type = typeField.control.value;
            setError(typeField, type ? null : 'Выберите тип');
            if (!type) {
                return;
            }
            lastType = type;
            await show(out, type);
        }, true);
    const show = async (out, type) => {
        const list = await api.typeLessThan(type);
        clear(out, h('p', {class: 'muted'}, `Найдено: ${pluralOrganizations(list.length)}`),
organizationsTable(list));
    };
    liveQueries.push(() => lastType && show(result, lastType));
}

// ---------- 4. Добавить сотрудника ----------
{
    let idField;
    idField = field('ID организации', idInput(() => idField));
    operationCard(
        'Добавить сотрудника',
        'Увеличивает количество сотрудников организации с указанным ID на одного.',
        [idField], 'Добавить сотрудника',
        async (out) => {
            const id = readId(idField);
            if (id === null) {
                return;
            }
            const o = await api.addEmployee(id);
            clear(out, h('div', {class: 'alert alert--success'},
                `В организации #${o.id} «${o.name}» теперь сотрудников: ${o.employeesCount}`));
            toast(`Сотрудник добавлен в организацию #${o.id}`, 'success');
        });
}

// ---------- 5. Поглощение ----------
{
    let absorberField;
    let absorbedField;
    absorberField = field('ID поглощающей', idInput(() => absorberField));
    absorberField.name = 'absorberId';
    absorbedField = field('ID поглощаемой', idInput(() => absorbedField));
    absorbedField.name = 'absorbedId';
    operationCard(
        'Поглощение организации',
        'Все сотрудники поглощаемой организации переходят в поглощающую (никто не увольняется), '
        + 'годовые обороты суммируются, поглощаемая организация удаляется.',
        [absorberField, absorbedField], 'Поглотить',
        async (out) => {
            const absorberId = readId(absorberField);
            const absorbedId = readId(absorbedField);
            if (absorberId === null || absorbedId === null) {
                return;
            }
            if (absorberId === absorbedId) {
                setError(absorbedField, 'Организация не может поглотить саму себя');
                return;
            }
            const r = await api.absorb(absorberId, absorbedId);
            clear(out, h('div', {class: 'alert alert--success'},
                `Организация #${r.absorber.id} «${r.absorber.name}» поглотила организацию #${r.absorbedId}. `
                + `Перешло сотрудников: ${r.transferredEmployees}, теперь их ${r.absorber.employeesCount}.`));
            toast(`Организация #${r.absorbedId} поглощена`, 'success');
        });
}

const rerun = debounce(() => liveQueries.forEach(q => q()?.catch?.(() => {
})), 300);
onChange((event) => {
    if (event.entity === 'ORGANIZATION' || event.entity === '*') {
        rerun();
```

```
    }
});
```

Листинг A.60 — src/main/resources/static/js/pages/view.js

```
import {api} from '../api.js';
import {deleteOrganizationWithConfirm, openEditDialog} from '../dialogs.js';
import {clear, formatDate, formatString, h, nullValue, pluralOrganizations, show, typeChip} from '../dom.js';
import {renderLayout} from '../layout.js';
import {numberInput, parseInteger} from '../organization-form.js';
import {debounce, onChange} from '../realtime.js';

renderLayout('view.html');

const app = document.getElementById('app');

// ---------- поиск по ID ----------

const idError = h('div', {class: 'error-text', id: 'id-error'});
const idInput = numberInput('integer', (msg) => {
    idError.textContent = msg;
});
idInput.id = 'org-id';
idInput.setAttribute('aria-describedby', 'id-error');
idInput.placeholder = 'Например, 1';

const searchForm = h('form', {class: 'card inline-form', novalidate: true},
    h('div', {class: 'field'}, h('label', {for: 'org-id'}, 'ID организации'), idInput, idError),
    h('button', {type: 'submit', class: 'btn btn--primary'}, 'Показать'));
searchForm.addEventListener('submit', (e) => {
    e.preventDefault();
    const parsed = parseInteger(idInput.value);
    let error = null;
    if (parsed.empty) {
        error = 'Введите ID';
    } else if (parsed.error) {
        error = parsed.error;
    } else if (parsed.value <= 0) {
        error = 'ID должен быть больше 0';
    }
    idError.textContent = error ?? '';
    searchForm.classList.toggle('invalid', Boolean(error));
    if (!error) {
        open(parsed.value);
    }
});

const content = h('div', {});
app.append(searchForm, content);

// ---------- отображение ----------

let currentId = null;
let deleted = false;

function open(id) {
    currentId = id;
    deleted = false;
    history.replaceState(null, '', `?id=${id}`);
    load();
}

async function load(flash = false) {
    const id = currentId;
    try {
        const [organization, coordinates, addresses] = await Promise.all([
            api.getOrganization(id), api.listCoordinates(), api.listAddresses()]);
        if (id !== currentId) {
            return;
        }
        render(organization,
            new Map(coordinates.map(c => [c.id, c.usages])),
            new Map(addresses.map(a => [a.id, a.usages])),
            flash);
    } catch (e) {
        if (id !== currentId) {
            return;
        }
```

```
        clear(content, h('div', {class: 'card'},
            h('div', {class: `alert ${e.status === 404 ? 'alert--warning' : 'alert--error'}`}, e.message),
            h('a', {href: 'index.html'}, '← К списку организаций')));
    }
}

function render(o, coordinatesUsages, addressUsages, flash) {
    const usage = (count) => count === undefined ? null
        : h('div', {class: 'hint'}, `Используется: ${pluralOrganizations(count)}`);
    const address = (title, a) => h('div', {class: 'related-card'},
        h('h3', {}, title),
        a ? h('dl', {class: 'props'},
            h('dt', {}, 'ID'), h('dd', {}, a.id),
            h('dt', {}, 'Индекс (zipCode)'), h('dd', {}, formatString(a.zipCode))) : h('div', {class: 'muted'},
'Не указан'),
        a ? usage(addressUsages.get(a.id)) : null);

    clear(content,
        h('div', {class: `card${flash ? ' flash' : ''}`},
            h('div', {class: 'page-head'},
                h('h2', {}, `#${o.id} · ${o.name}`),
                h('div', {class: 'btn-row'},
                    h('button', {type: 'button', class: 'btn btn--primary', onclick: edit}, 'Изменить'),
                    h('button', {type: 'button', class: 'btn btn--danger', onclick: () => remove(o)},
'Удалить'))),
            h('dl', {class: 'props'},
                h('dt', {}, 'ID'), h('dd', {}, o.id),
                h('dt', {}, 'Название (name)'), h('dd', {}, o.name),
                h('dt', {}, 'Полное название (fullName)'), h('dd', {}, formatString(o.fullName)),
                h('dt', {}, 'Тип (type)'), h('dd', {}, typeChip(o.type)),
                h('dt', {}, 'Дата создания'), h('dd', {}, formatDate(o.creationDate)),
                h('dt', {}, 'Количество сотрудников'), h('dd', {}, o.employeesCount),
                h('dt', {}, 'Годовой оборот'), h('dd', {}, show(o.annualTurnover)),
                h('dt', {}, 'Рейтинг'), h('dd', {}, o.rating ?? nullValue()))),
        h('div', {class: 'card'},
            h('h2', {}, 'Связанные объекты'),
            h('div', {class: 'relations-grid'},
                h('div', {class: 'related-card'},
                    h('h3', {}, 'Координаты'),
                    h('dl', {class: 'props'},
                        h('dt', {}, 'ID'), h('dd', {}, o.coordinates.id),
                        h('dt', {}, 'X'), h('dd', {}, o.coordinates.x),
                        h('dt', {}, 'Y'), h('dd', {}, o.coordinates.y)),
                    usage(coordinatesUsages.get(o.coordinates.id))),
                address('Официальный адрес', o.officialAddress),
                address('Почтовый адрес', o.postalAddress))));
}

function showDeleted() {
    deleted = true;
    clear(content, h('div', {class: 'card'},
        h('div', {class: 'alert alert--warning'}, `Организация #${currentId} была удалена.`),
        h('a', {href: 'index.html'}, '← К списку организаций')));
}

async function edit() {
    if (await openEditDialog(currentId)) {
        load(true);
    }
}

async function remove(organization) {
    if (await deleteOrganizationWithConfirm(organization)) {
        showDeleted();
    }
}

// ---------- синхронизация ----------

const reload = debounce((flash) => load(flash));

onChange((event) => {
    if (currentId === null || deleted) {
        return;
    }
    const affectsCurrent = event.entity === 'ORGANIZATION' && event.ids.includes(currentId);
    if (affectsCurrent && event.change === 'DELETED') {
        showDeleted();
```

```
    } else {
        // Изменение самой организации или счётчиков использования её координат/адресов
        reload(affectsCurrent);
    }
});

const initialId = parseInteger(new URLSearchParams(location.search).get('id') ?? '');
if (initialId.value > 0) {
    idInput.value = String(initialId.value);
    open(initialId.value);
} else {
    idInput.focus();
}
```

Листинг A.61 — src/main/resources/static/css/app.css

```
:root {
    --bg: #f4f5f7;
    --surface: #ffffff;
    --surface-alt: #f9fafb;
    --border: #dfe3e8;
    --border-strong: #c5cbd3;
    --text: #1c2230;
    --muted: #667085;
    --accent: #2563eb;
    --accent-hover: #1d4ed8;
    --accent-soft: #e8efff;
    --danger: #c9372c;
    --danger-hover: #ae2e24;
    --danger-soft: #fdecea;
    --success: #19803d;
    --success-soft: #e7f6ec;
    --warning-soft: #fff6e0;
    --warning-border: #f0c25a;
    --radius: 8px;
    --shadow: 0 1px 2px rgba(16, 24, 40, .06), 0 1px 3px rgba(16, 24, 40, .1);
    --shadow-lg: 0 12px 32px rgba(16, 24, 40, .18);
}

* {
    box-sizing: border-box;
}

[hidden] {
    display: none !important;
}

body {
    margin: 0;
    background: var(--bg);
    color: var(--text);
    font: 14px/1.45 system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
}

a {
    color: var(--accent);
    text-decoration: none;
}

a:hover {
    text-decoration: underline;
}

h1 {
    font-size: 22px;
    margin: 0 0 4px;
}

h2 {
    font-size: 16px;
    margin: 0 0 12px;
}

.muted {
    color: var(--muted);
}

/* ---------- Шапка и меню ---------- */
```

```
.app-header {
    background: #1f2937;
    color: #f9fafb;
    position: sticky;
    top: 0;
    z-index: 20;
}

.app-header__inner {
    max-width: 1500px;
    margin: 0 auto;
    padding: 0 20px;
    display: flex;
    align-items: center;
    gap: 24px;
    min-height: 54px;
    flex-wrap: wrap;
}

.brand {
    font-weight: 700;
    font-size: 15px;
    color: #fff;
    white-space: nowrap;
}

.brand:hover {
    text-decoration: none;
}

.menu {
    display: flex;
    gap: 4px;
    flex-wrap: wrap;
    flex: 1;
}

.menu a {
    color: #cbd5e1;
    padding: 7px 12px;
    border-radius: 6px;
    white-space: nowrap;
}

.menu a:hover {
    background: rgba(255, 255, 255, .08);
    color: #fff;
    text-decoration: none;
}

.menu a.active {
    background: rgba(255, 255, 255, .14);
    color: #fff;
}

.live-status {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    font-size: 12px;
    color: #cbd5e1;
    white-space: nowrap;
}

.live-status::before {
    content: "";
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background: #9ca3af;
}

.live-status.online::before {
    background: #34d399;
}

.live-status.offline::before {
```

```
    background: #f87171;
}

/* ---------- Каркас страницы ---------- */

.page {
    max-width: 1500px;
    margin: 0 auto;
    padding: 24px 20px 48px;
}

.page--narrow {
    max-width: 920px;
}

.page-head {
    display: flex;
    align-items: flex-end;
    justify-content: space-between;
    gap: 16px;
    margin-bottom: 18px;
    flex-wrap: wrap;
}

.card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
    padding: 18px 20px;
}

.card + .card {
    margin-top: 16px;
}

.grid-2 {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(380px, 1fr));
    gap: 16px;
}

.grid-2 > .card + .card {
    margin-top: 0;
}

.grid-2--fixed {
    grid-template-columns: repeat(2, minmax(0, 1fr));
}

.grid-2 > .wide {
    grid-column: 1 / -1;
}

@media (max-width: 900px) {
    .grid-2--fixed {
        grid-template-columns: minmax(0, 1fr);
    }
}

/* ---------- Кнопки ---------- */

.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 6px;
    border: 1px solid var(--border-strong);
    background: var(--surface);
    color: var(--text);
    border-radius: 6px;
    padding: 7px 14px;
    font: inherit;
    font-weight: 500;
    cursor: pointer;
    white-space: nowrap;
    text-decoration: none;
}
```

```
.btn:hover {
    background: var(--surface-alt);
    text-decoration: none;
}

.btn:disabled {
    opacity: .55;
    cursor: not-allowed;
}

.btn--primary {
    background: var(--accent);
    border-color: var(--accent);
    color: #fff;
}

.btn--primary:hover {
    background: var(--accent-hover);
}

.btn--danger {
    background: var(--danger);
    border-color: var(--danger);
    color: #fff;
}

.btn--danger:hover {
    background: var(--danger-hover);
}

.btn--ghost {
    border-color: transparent;
    background: transparent;
}

.btn--sm {
    padding: 3px 9px;
    font-size: 13px;
}

.btn-row {
    display: flex;
    gap: 8px;
    flex-wrap: wrap;
    align-items: center;
}

/* ---------- Формы ---------- */

.form-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    gap: 14px 18px;
}

.field {
    display: flex;
    flex-direction: column;
    gap: 4px;
    min-width: 0;
}

.field > label,
.field-label {
    font-weight: 600;
    font-size: 13px;
}

.req {
    color: var(--danger);
}

.hint {
    font-size: 12px;
    color: var(--muted);
}
```

```
label.check {
    display: inline-flex;
    align-items: center;
    gap: 4px;
    cursor: pointer;
    user-select: none;
}

input:disabled {
    background: var(--surface-alt);
    color: var(--muted);
}

input[type="text"],
input[type="search"],
select {
    font: inherit;
    border: 1px solid var(--border-strong);
    border-radius: 6px;
    padding: 7px 10px;
    background: #fff;
    color: var(--text);
    min-width: 0;
    width: 100%;
}

input:focus,
select:focus {
    outline: 2px solid rgba(37, 99, 235, .35);
    outline-offset: 0;
    border-color: var(--accent);
}

.invalid input,
.invalid select,
input.invalid,
select.invalid {
    border-color: var(--danger);
    background: #fffafa;
}

.error-text {
    color: var(--danger);
    font-size: 12px;
    min-height: 0;
}

.error-text:empty {
    display: none;
}

fieldset.group {
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 12px 14px 14px;
    margin: 0;
    min-width: 0;
}

fieldset.group > legend {
    font-weight: 600;
    padding: 0 6px;
}

.form-section-title {
    font-weight: 700;
    margin: 18px 0 10px;
    color: var(--muted);
    text-transform: uppercase;
    font-size: 12px;
    letter-spacing: .04em;
}

.segmented {
    display: inline-flex;
    border: 1px solid var(--border-strong);
    border-radius: 6px;
    overflow: hidden;
```

```
    margin-bottom: 10px;
    flex-wrap: wrap;
}

.segmented label {
    padding: 5px 12px;
    cursor: pointer;
    user-select: none;
    border-right: 1px solid var(--border-strong);
    font-size: 13px;
}

.segmented label:last-child {
    border-right: none;
}

.segmented input {
    position: absolute;
    opacity: 0;
    pointer-events: none;
}

.segmented label:has(input:checked) {
    background: var(--accent);
    color: #fff;
}

.segmented label:has(input:disabled) {
    color: var(--muted);
    cursor: not-allowed;
    background: var(--surface-alt);
}

.segmented label:has(input:focus-visible) {
    outline: 2px solid rgba(37, 99, 235, .45);
    outline-offset: -2px;
}

.relations-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 14px;
}

.form-actions {
    display: flex;
    gap: 8px;
    justify-content: flex-end;
    margin-top: 18px;
    flex-wrap: wrap;
}

.alert {
    border-radius: 6px;
    padding: 10px 12px;
    margin-bottom: 14px;
    border: 1px solid;
}

.alert:empty {
    display: none;
}

.alert--error {
    background: var(--danger-soft);
    border-color: #f3b8b2;
    color: #8a1f17;
}

.alert--success {
    background: var(--success-soft);
    border-color: #a6dbb7;
    color: #135c2c;
}

.alert--warning {
    background: var(--warning-soft);
    border-color: var(--warning-border);
```

```
    color: #7a5200;
}

.alert ul {
    margin: 4px 0 0;
    padding-left: 18px;
}

/* ---------- Таблица ---------- */

.filters {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
    gap: 10px 14px;
    align-items: start;
}

.filters .btn-row {
    grid-column: 1 / -1;
    justify-content: flex-end;
}

.table-wrap {
    overflow-x: auto;
    border: 1px solid var(--border);
    border-radius: var(--radius);
    background: var(--surface);
}

table.data {
    border-collapse: collapse;
    width: 100%;
    font-size: 13px;
}

table.data th,
table.data td {
    padding: 7px 8px;
    border-bottom: 1px solid var(--border);
    text-align: left;
    vertical-align: top;
}

table.data th {
    background: var(--surface-alt);
    font-weight: 600;
    position: sticky;
    top: 0;
    vertical-align: bottom;
}

table.data th.sortable {
    cursor: pointer;
    user-select: none;
}

table.data th.sortable:hover {
    background: #eef1f5;
}

table.data th .sort-mark {
    color: var(--muted);
    margin-left: 4px;
    font-size: 11px;
}

table.data th.sorted .sort-mark {
    color: var(--accent);
}

table.data tbody tr:hover {
    background: #fafbfd;
}

table.data tbody tr:last-child td {
    border-bottom: none;
}
```

```
table.data td.num {
    text-align: right;
    font-variant-numeric: tabular-nums;
    white-space: nowrap;
}

table.data td.nowrap {
    white-space: nowrap;
}

table.data th.num {
    text-align: right;
}

/* Кнопки действий всегда видны, даже когда таблица прокручивается по горизонтали */
table.data td.actions,
table.data th.actions {
    white-space: nowrap;
    text-align: right;
    position: sticky;
    right: 0;
    background: var(--surface);
    box-shadow: -8px 0 8px -8px rgba(16, 24, 40, .25);
}

table.data th.actions {
    background: var(--surface-alt);
    z-index: 2;
}

table.data tbody tr:hover td.actions {
    background: #fafbfd;
}

table.data .empty {
    text-align: center;
    color: var(--muted);
    padding: 28px;
}

.null {
    color: #a0a7b4;
}

.chip {
    display: inline-block;
    padding: 1px 6px;
    border-radius: 10px;
    background: var(--accent-soft);
    color: #1e40af;
    font-size: 11px;
    font-weight: 600;
    white-space: nowrap;
}

.chip--GOVERNMENT {
    background: #e6f4ff;
    color: #0b5394;
}

.chip--TRUST {
    background: #f1ecff;
    color: #5b2bb5;
}

.chip--PRIVATE_LIMITED_COMPANY {
    background: #e9f7ef;
    color: #18703a;
}

@keyframes flash {
    from {
        background: #fff3b0;
    }
    to {
        background: transparent;
    }
}
```

```
.flash {
    animation: flash 1.8s ease-out;
}

.toolbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    margin: 14px 0 10px;
    flex-wrap: wrap;
}

.pager {
    display: flex;
    align-items: center;
    gap: 6px;
    flex-wrap: wrap;
}

.pager select {
    width: auto;
    padding: 4px 8px;
}

.pager .page-info {
    margin: 0 6px;
    color: var(--muted);
}

/* ---------- Просмотр объекта ---------- */

.props {
    display: grid;
    grid-template-columns: max-content 1fr;
    gap: 6px 18px;
    margin: 0;
}

.props dt {
    color: var(--muted);
}

.props dd {
    margin: 0;
    font-weight: 500;
    word-break: break-word;
}

.related-card {
    background: var(--surface-alt);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 12px 14px;
}

.related-card h3 {
    font-size: 14px;
    margin: 0 0 8px;
}

.inline-form {
    display: flex;
    gap: 8px;
    align-items: flex-start;
    flex-wrap: wrap;
}

.inline-form .field {
    min-width: 180px;
}

.inline-form .btn {
    margin-top: 22px;
}

.result {
```

```
    margin-top: 12px;
}

.result:empty {
    display: none;
}

.big-number {
    font-size: 28px;
    font-weight: 700;
}

/* ---------- Диалоги ---------- */

dialog.modal {
    border: none;
    border-radius: 10px;
    padding: 0;
    width: min(980px, calc(100vw - 32px));
    max-height: calc(100vh - 32px);
    box-shadow: var(--shadow-lg);
    color: var(--text);
}

dialog.modal--small {
    width: min(480px, calc(100vw - 32px));
}

dialog.modal::backdrop {
    background: rgba(15, 23, 42, .45);
}

.modal__head {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 14px 20px;
    border-bottom: 1px solid var(--border);
    position: sticky;
    top: 0;
    background: var(--surface);
    z-index: 1;
}

.modal__head h2 {
    margin: 0;
}

.modal__body {
    padding: 18px 20px 20px;
}

.icon-btn {
    border: none;
    background: transparent;
    font-size: 22px;
    line-height: 1;
    cursor: pointer;
    color: var(--muted);
    padding: 2px 6px;
    border-radius: 4px;
}

.icon-btn:hover {
    background: var(--surface-alt);
    color: var(--text);
}

/* ---------- Уведомления ---------- */

.toasts {
    position: fixed;
    right: 16px;
    bottom: 16px;
    display: flex;
    flex-direction: column;
    gap: 8px;
    z-index: 1000;
```

```
    max-width: min(420px, calc(100vw - 32px));
}

.toast {
    background: #111827;
    color: #f9fafb;
    padding: 10px 14px;
    border-radius: 8px;
    box-shadow: var(--shadow-lg);
    border-left: 4px solid #60a5fa;
    animation: toast-in .2s ease-out;
}

.toast--success {
    border-left-color: #34d399;
}

.toast--error {
    border-left-color: #f87171;
}

@keyframes toast-in {
    from {
        transform: translateY(8px);
        opacity: 0;
    }
    to {
        transform: none;
        opacity: 1;
    }
}

@media (max-width: 640px) {
    .grid-2 {
        grid-template-columns: 1fr;
    }

    .props {
        grid-template-columns: 1fr;
    }

    .props dd {
        margin-bottom: 6px;
    }
}
```